



**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN**

**ĐỒ ÁN MÔN HỌC
THỊ GIÁC MÁY TÍNH**

**TRÍCH XUẤT THÔNG TIN
BIÊN LAI TIẾNG VIỆT**

*So sánh kiến trúc Donut (OCR-free)
và VietOCR + LayoutXLM (OCR-based)*

Giảng viên hướng dẫn:

ThS. Nguyễn Đình Quý

Bộ môn:

Khoa học máy tính

Sinh viên thực hiện:

Trần Phùng Đức Anh - 0204168

Nguyễn Minh Năng - 0211668

Nguyễn Việt Hùng - 0208768

Lớp:

68CS2

Hà Nội, 2026

Lời cảm ơn

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Nguyễn Đình Quý, giảng viên bộ môn Khoa học máy tính, Khoa Công nghệ thông tin, Trường Đại học Xây dựng Hà Nội. Thầy đã tận tình hướng dẫn, định hướng khoa học và cung cấp những chỉ dẫn vô cùng quý báu trong suốt quá trình nghiên cứu và hoàn thành đồ án môn học Thị giác máy tính này.

Chúng em cũng xin gửi lời cảm ơn đến các thầy cô giáo trong Khoa Công nghệ thông tin đã truyền đạt những kiến thức nền tảng bổ ích, tạo điều kiện thuận lợi nhất để chúng em thực hiện đề tài.

Cuối cùng, xin cảm ơn gia đình, bạn bè và tập thể lớp 68CS2 đã luôn động viên, hỗ trợ và đóng góp ý kiến để nhóm hoàn thiện báo cáo đồ án này một cách tốt nhất. Do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp và chỉ bảo của quý thầy cô.

Chúng em xin chân thành cảm ơn!

Hà Nội, năm 2026

Nhóm sinh viên thực hiện

Mục lục

Lời cảm ơn	1
Danh mục hình ảnh	5
Danh mục bảng biểu	6
Danh mục từ viết tắt	7
Tóm tắt	8
1 Tổng quan đề tài	9
1.1 Đặt vấn đề	9
1.2 Câu hỏi nghiên cứu	9
1.3 Mục tiêu và chỉ số đo lường	9
1.4 Phạm vi đề tài	10
1.5 Đóng góp thực tiễn	10
1.6 Cấu trúc báo cáo	10
2 Cơ sở lý thuyết	12
2.1 Bài toán trích xuất thông tin tài liệu	12
2.2 Các công trình nghiên cứu liên quan	12
2.3 Nhận dạng ký tự quang học (OCR)	14
2.3.1 Phát hiện văn bản với PaddleOCR	14
2.3.2 Nhận dạng văn bản tiếng Việt với VietOCR	14
2.4 Mô hình Donut (OCR-free)	14
2.4.1 Kiến trúc tổng quan	15
2.4.2 Phân biệt khái niệm Warm-up trong Donut	15
2.5 Mô hình LayoutXLM (OCR-based)	15
2.5.1 Nhúng đa phương thức (Multimodal Embedding)	16
2.5.2 Spatial-aware Self-Attention	16
2.5.3 Gán nhãn BIO (BIO Tagging)	16
2.6 Các độ đo đánh giá và quy ước xử lý rỗng	16
2.6.1 Exact Match (EM)	17
2.6.2 Normalized Edit Similarity (NES)	17
2.6.3 Character Error Rate (CER)	17
2.6.4 Hai chế độ tổng hợp kết quả	18

3	Tập dữ liệu	19
3.1	Nguồn dữ liệu	19
3.2	Schema JSONL thống nhất	19
3.3	Chia dữ liệu và chống rò rỉ	19
3.4	Mức annotation và dữ liệu huấn luyện	20
3.5	Giá trị ground-truth rỗng trên test	20
3.6	Kiểm tra hợp lệ	20
4	Tiền xử lý dữ liệu	21
4.1	Tiền xử lý ảnh (Image Preprocessing)	21
4.2	Tăng cường dữ liệu (Data Augmentation)	21
4.3	Chuẩn hoá văn bản (Text Normalization)	22
4.4	Xây dựng OCR Cache	22
4.5	Trạng thái khảo sát tiền xử lý OCR	23
5	Phương pháp thực hiện	24
5.1	Tổng quan kiến trúc hệ thống	24
5.2	Baseline: OCR + Trích xuất dựa trên luật (Rule-based)	24
5.2.1	Pipeline tổng quan	25
5.2.2	Quy tắc trích xuất cụ thể	25
5.3	Donut: End-to-End OCR-free	26
5.3.1	Chuẩn bị dữ liệu	26
5.3.2	Cấu hình huấn luyện	27
5.3.3	Suy luận (Inference)	27
5.4	VietOCR + LayoutXLM: OCR-based Layout-aware	28
5.4.1	Chuẩn bị dữ liệu	28
5.4.2	Cấu hình huấn luyện	28
5.4.3	Suy luận và hậu xử lý (Inference & Post-processing)	30
5.5	Hậu xử lý chung (Post-processing)	30
6	Thiết lập thực nghiệm	31
6.1	Trạng thái tệp kết quả dự đoán	31
6.2	Môi trường và cấu hình	31
6.3	Phạm vi dữ liệu của từng pipeline	32
6.4	Quy trình suy luận và đánh giá	32
6.5	Phạm vi phép đo latency	33
6.6	Trạng thái bằng chứng huấn luyện	33
7	Kết quả thực nghiệm và đánh giá	35
7.1	Kết quả trên toàn bộ mẫu và trên trường có mặt	35

7.2	Phân tích theo trường	37
7.3	Latency của các thành phần đã ghi nhận	38
7.4	Phân tích hiệu năng Donut và hiện tượng suy sụp	39
7.5	Phân tích lỗi có thể tái lập	39
7.6	Oracle OCR và câu hỏi về ảnh hưởng của OCR	40
7.7	Demo ứng dụng	41
8	Kết luận và hướng phát triển	42
8.1	Trả lời câu hỏi nghiên cứu	42
8.2	Các yếu tố ảnh hưởng đến độ tin cậy của kết quả	43
8.3	Đóng góp và hạn chế	43
8.4	Hướng phát triển	44
	Tài liệu tham khảo	46
A	Mẫu dữ liệu JSONL thống nhất	47
B	Cấu trúc mã nguồn	48
C	Cấu hình huấn luyện đầy đủ	49
C.1	Cấu hình DONUT (donut.yaml)	49
C.2	Cấu hình LAYOUTXLM (layoutxlm.yaml)	50
C.3	Cấu hình luật heuristic (baseline.yaml)	51
D	Mẫu kết quả trích xuất thực tế	53
E	Giao diện Demo Gradio	54
E.1	Hướng dẫn cài đặt và chạy demo	54

Danh sách hình vẽ

2.1	Kiến trúc tổng quan mô hình DONUT: Vision Encoder (Swin Transformer) trích xuất đặc trưng thị giác z và Text Decoder (mBART) sinh chuỗi kết quả tự hỏi quy [4].	15
5.1	Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.	24
7.1	So sánh Exact Match trên các mẫu có ground-truth khác rỗng. Hình được sinh từ metrics JSON.	38
E.1	Giao diện Gradio trong một lượt chạy thực tế, so sánh kết quả của Baseline, DONUT và VietOCR + LAYOUTXLM.	54

Danh sách bảng

1.1	Mô tả các trường thông tin trong phạm vi trích xuất.	10
2.1	So sánh đặc tính và giả thuyết kiểm chứng của ba hướng tiếp cận.	13
2.2	Hệ thống nhãn Begin-Inside-Outside — Lược đồ gán nhãn chuỗi (BIO) cho 4 trường thông tin trích xuất.	17
3.1	Các trường chính trong schema JSONL.	19
3.2	Phân bố mẫu trong các tệp processed.	20
3.3	Mức annotation theo split.	20
3.4	Số mẫu có ground-truth rỗng trên 234 mẫu test.	20
4.1	Các phép tăng cường dữ liệu được cài đặt cho hai mô hình.	21
4.2	Ví dụ chuẩn hoá văn bản theo trường thông tin.	22
5.1	Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.	25
5.2	Cấu hình huấn luyện mô hình DONUT.	27
5.3	Ví dụ tối giản về gán nhãn BIO cho một chuỗi OCR.	29
5.4	Cấu hình huấn luyện mô hình LAYOUTXLM.	29
6.1	Trạng thái bằng chứng của các prediction artifact đã khóa bằng SHA-256.	31
6.2	Cấu hình huấn luyện được khai báo cho Donut và LayoutXLM.	32
6.3	Số mẫu có thể sử dụng ở từng giai đoạn.	32
7.1	Kết quả trên toàn bộ 234 mẫu kiểm thử của các artifact hợp lệ. Macro là trung bình không trọng số của bốn trường.	35
7.2	Kết quả present-only của các artifact hợp lệ: mỗi trường chỉ được chấm trên mẫu có ground-truth khác rỗng.	36
7.3	Độ phủ prediction và số mẫu đánh giá. Số mẫu present-only lần lượt là 224, 216, 217 và 223 cho bốn trường.	36
7.4	Paired bootstrap present-only cho chênh lệch LayoutXLM trừ Baseline, seed 20260620, 50,000 lần lấy mẫu.	37
7.5	Phân tích độ nhạy present-only sau khi loại bốn mẫu có sai lệch hệ tọa độ ảnh/OCR ($n = 230$).	37
7.6	Latency hậu xử lý từ OCR cache. Phép đo bắt đầu sau khi OCR cache đã sẵn sàng; chưa bao gồm thời gian chạy PaddleOCR + VietOCR từ ảnh thô	38
7.7	Latency suy luận trực tiếp từ ảnh thô. Phép đo này không so sánh ngang hàng với bảng post-OCR cache vì phạm vi đo hoàn toàn khác nhau.	38
7.8	Thông tin bổ sung về artifact Donut (generation length analysis).	39
7.9	Phân loại lỗi được sinh tự động từ tệp kết quả dự đoán.	40
7.10	So sánh LAYOUTXLM với OCR thực tế và Oracle OCR trên 168 mẫu MC-OCR có ground-truth OCR. Các số liệu là present-only; số mẫu theo bốn trường lần lượt là 158, 151, 151 và 157.	41
D.1	Ví dụ kết quả trích xuất thực tế so với nhãn ground-truth trên tập test.	53

Danh mục từ viết tắt

BIO Begin-Inside-Outside — Lược đồ gán nhãn chuỗi. [11](#), [15](#), [22](#), [26](#), [27](#)

NFC Unicode Normalization Form Composed — Dạng chuẩn hoá Unicode tổ hợp. [27](#), [28](#)

OCR Optical Character Recognition — Nhận dạng ký tự quang học. [7](#), [10](#), [22](#), [26](#)

Tóm tắt

Đồ án xây dựng hệ thống trích xuất tên cửa hàng, ngày giao dịch, tổng tiền và địa chỉ từ ảnh biên lai tiếng Việt. Ba phương pháp được triển khai: Baseline dùng OCR kết hợp luật, DONUT sinh chuỗi trực tiếp từ ảnh, và LAYOUTXLM phân loại token dựa trên văn bản, bố cục và ảnh. Bộ dữ liệu có 1.559 mẫu; Donut dùng toàn bộ 1.091 mẫu train, còn LayoutXLM dùng 784 mẫu train có hộp bao. Cả ba phương pháp đều được đánh giá trên cùng 234 mẫu test.

Để tránh điểm số bị ảnh hưởng bởi nhãn rỗng, báo cáo trình bày cả kết quả trên toàn bộ mẫu và kết quả present-only, chỉ xét các mẫu có ground-truth khác rỗng của từng trường. Trên present-only, LAYOUTXLM đạt kết quả tốt nhất với Macro EM 42,96% và Macro NES 63,89%; Baseline đạt 40,41% và 55,47%; còn DONUT gặp hiện tượng sụp đổ (model collapse, Macro EM 0,00%) do giới hạn dữ liệu huấn luyện, do đó chưa phản ánh được tiềm năng của kiến trúc OCR-free. Chênh lệch giữa LayoutXLM và Baseline còn khiêm tốn, nhưng Oracle OCR trên 168 mẫu MC-OCR có ground-truth OCR cho thấy Macro EM của LayoutXLM tăng từ 49,72% khi dùng OCR thực tế lên 95,55% khi dùng OCR hoàn hảo. Kết quả này chỉ ra OCR là nút thắt cổ chai lớn, đồng thời báo cáo công bố rõ các giới hạn về annotation không đồng nhất, bốn mẫu LayoutXLM có sai lệch hệ tọa độ, tiêu chí checkpoint lịch sử, thiếu log huấn luyện đầy đủ và phạm vi latency chưa phải end-to-end từ ảnh thô.

Từ khoá: trích xuất thông tin, biên lai, Donut, LayoutXLM, OCR, tiếng Việt.

1. Tổng quan đề tài

Báo cáo này tập trung vào bài toán trích xuất thông tin then chốt từ biên lai tiếng Việt. Dưới đây là bối cảnh đề tài, câu hỏi nghiên cứu, mục tiêu đo lường, phạm vi triển khai cùng cấu trúc tổng quát của đề án.

1.1. Đặt vấn đề

Biên lai tiếng Việt là loại tài liệu khó xử lý tự động vì ảnh thường được chụp bằng điện thoại, chất lượng không ổn định, bố cục thay đổi theo từng cửa hàng và văn bản có dấu tiếng Việt dễ bị OCR nhận sai. Trong đề án này, nhóm tập trung vào bốn trường thông tin thường cần nhập liệu lại: tên cửa hàng, ngày giao dịch, tổng tiền và địa chỉ.

Hiện tại có hai hướng tiếp cận chính cho bài toán này:

- **Pipeline dựa trên OCR** (như kết hợp PaddleOCR/VietOCR với mô hình ngôn ngữ hiểu bố cục LAYOUTXLM [11]): Tách riêng giai đoạn nhận dạng chữ và giai đoạn phân loại thực thể cấp token.
- **Pipeline không dùng OCR trung gian** (như mô hình sinh end-to-end DONUT [4]): Dùng mạng Transformer sinh trực tiếp chuỗi có cấu trúc từ ảnh đầu vào, không cần module nhận dạng văn bản trung gian.

So sánh hai hướng tiếp cận này trên cùng tập kiểm thử sẽ làm rõ ưu nhược điểm của từng mô hình trong thực tế. Dù vậy, để so sánh hoàn toàn công bằng giữa các kiến trúc, cần kiểm soát cả lượng dữ liệu và mức độ gán nhãn đồng nhất.

1.2. Câu hỏi nghiên cứu

Đề án đặt ra bốn câu hỏi nghiên cứu chính để định hướng thực nghiệm và đánh giá:

- **RQ1:** Trong điều kiện dữ liệu thực tế, mô hình dựa trên OCR LAYOUTXLM cải thiện hiệu năng như thế nào so với Baseline dựa trên luật?
- **RQ2:** Mô hình không dùng OCR trung gian DONUT hoạt động ra sao khi huấn luyện trên tập biên lai tiếng Việt nhỏ và không đồng nhất?
- **RQ3:** Chất lượng OCR ảnh hưởng đến hiệu năng của LAYOUTXLM ở mức nào?
- **RQ4:** Chi phí suy luận của từng pipeline khác nhau ra sao trong phạm vi thành phần đã được ghi nhận?

1.3. Mục tiêu và chỉ số đo lường

Đề án hướng tới thiết kế, huấn luyện và đánh giá ba phương pháp trích xuất 4 trường thông tin quan trọng từ biên lai tiếng Việt: tên cửa hàng (**store_name**), ngày tháng (**date**), tổng tiền (**total**), và địa chỉ (**address**).

Hiệu năng các phương pháp được đo trên cùng tập kiểm thử độc lập qua bốn chỉ số:

- **Exact Match (EM):** Tỷ lệ dự đoán khớp chính xác tuyệt đối từng ký tự với nhãn thật sau chuẩn hóa.
- **Normalized Edit Similarity (NES):** Đo độ tương đồng chuỗi ký tự dựa trên khoảng cách Levenshtein chuẩn hóa.
- **Character Error Rate (CER):** Tỷ lệ lỗi ký tự, đo lường chi tiết sai lệch chữ viết.
- **Latency thành phần:** Thời gian được ghi trong prediction artifact; phạm vi bắt đầu khác nhau giữa pipeline dựa trên OCR và Donut.

1.4. Phạm vi đề tài

Đề tài tập trung vào việc đánh giá khả năng trích xuất thông tin trên 4 trường dữ liệu chính được mô tả trong [Bảng 1.1](#).

Bảng 1.1: Mô tả các trường thông tin trong phạm vi trích xuất.

Trường	Định dạng chuẩn hóa	Ví dụ thực tế
store_name	Chuỗi Unicode NFC	CIRCLE K
date	Chuỗi định dạng YYYY-MM-DD	2021-03-15
total	Chuỗi số nguyên không dấu	115000
address	Chuỗi Unicode NFC	123 Nguyễn Trãi, Thanh Xuân

Ngoài phạm vi thực hiện (Out of Scope):

- Hệ thống không trích xuất danh sách chi tiết các mặt hàng mua sắm (items list).
- Không trích xuất mã số thuế (MST), số điện thoại, email, website hoặc số hóa đơn.
- Chỉ làm việc trực tiếp trên dữ liệu ảnh chụp (JPG, PNG), không xử lý file PDF số hóa.

1.5. Đóng góp thực tiễn

Đóng góp chính của đồ án là một pipeline có thể chạy lại được ở mức artifact: dữ liệu được chuẩn hóa về JSONL, ba hướng trích xuất cùng xuất ra một schema, bộ evaluator chấm EM/NES/CER thống nhất, và các bảng trong báo cáo được sinh từ prediction artifact thay vì nhập tay.

Để phục vụ việc kiểm chứng và phát triển tiếp theo, toàn bộ mã nguồn của đồ án được phát triển và lưu trữ công khai tại địa chỉ GitHub: <https://github.com/Duc-AnhTp/receipt-vn-ie>.

1.6. Cấu trúc báo cáo

Báo cáo gồm các chương sau:

- **Mục 2 (Cơ sở lý thuyết):** Trình bày bài toán trích xuất thông tin, lý thuyết về OCR, kiến trúc Donut, LayoutXLM và các công trình liên quan.
- **Mục 3 (Thu thập và xây dựng tập dữ liệu):** Mô tả các nguồn dữ liệu, quy tắc gán nhãn, schema thống nhất và quy trình loại mẫu trùng lặp.

- **Mục 4 (Tiền xử lý dữ liệu):** Chi tiết các bước xử lý hình ảnh, tăng cường dữ liệu, chuẩn hóa văn bản, xây dựng OCR cache và trạng thái khảo sát ablation.
- **Mục 5 (Phương pháp thực hiện):** Thiết kế chi tiết và cấu hình triển khai của cả ba pipeline.
- **Mục 6 (Thực nghiệm):** Cấu hình, phạm vi dữ liệu, protocol đánh giá và giới hạn bằng chứng huấn luyện.
- **Mục 7 (Kết quả thực nghiệm):** Thảo luận kết quả all-samples, present-only, latency thành phần và phân tích lỗi.
- **Mục 8 (Kết luận):** Trả lời các câu hỏi nghiên cứu, rút ra hạn chế và hướng phát triển tiếp theo.

2. Cơ sở lý thuyết

Để giải thích cơ sở lựa chọn các phương pháp Baseline, Donut và LayoutXLM, nội dung dưới đây tóm tắt các khái niệm nền tảng: trích xuất thông tin tài liệu, OCR, mô hình không dùng OCR trung gian, mô hình dựa trên OCR và các thước đo đánh giá chuỗi.

2.1. Bài toán trích xuất thông tin tài liệu

Trích xuất thông tin tài liệu (Document Information Extraction - DIE) hay trích xuất thông tin then chốt (Key Information Extraction - KIE) là bài toán nhận dạng và lấy ra các thực thể thông tin có cấu trúc dạng cặp khóa-giá trị (key-value pairs) từ hình ảnh tài liệu bán cấu trúc (biên lai, hóa đơn, tờ khai...).

Đầu vào là một ảnh biên lai $I \in \mathbb{R}^{H \times W \times C}$. Đầu ra mong muốn là một tập thực thể: $Y = \{(K_1, V_1), (K_2, V_2), \dots, (K_M, V_M)\}$, trong đó K_j tương ứng với một trường thông tin đích (ví dụ: **store_name**, **date**, **total**, **address**) và V_j là văn bản chuẩn hóa được trích xuất. Mô hình cần vừa nhận diện được ký tự, vừa hiểu được quan hệ không gian và ngữ nghĩa giữa các vùng chữ để liên kết chúng đúng.

2.2. Các công trình nghiên cứu liên quan

Với biên lai, chỉ nhận dạng được chữ là chưa đủ. Mô hình còn phải xác định dòng nào là tổng tiền, dòng nào là địa chỉ và khi nào một thực thể bị tách qua nhiều dòng. [Bảng 2.1](#) so sánh ba hướng tiếp cận được khảo sát trong đồ án cùng giả thuyết thực nghiệm.

Các nghiên cứu và công trình liên quan trực tiếp đến đồ án gồm:

- **MC-OCR 2021:** Cuộc thi xử lý biên lai chụp di động do Zalo AI tổ chức [12], cung cấp bộ dữ liệu biên lai tiếng Việt thực tế làm nguồn dữ liệu chính cho đồ án. Các giải pháp top đầu thường kết hợp bộ phát hiện chữ (DBNet, YOLO) với bộ nhận dạng mạnh, cùng các luật hậu xử lý phức tạp.
- **CORD:** Bộ dữ liệu biên lai quốc tế Consolidated Receipt Dataset [9], được dùng làm benchmark chính cho các kiến trúc hiểu tài liệu end-to-end.
- **LayoutLM và LayoutLMv3:** Các mô hình tiền huấn luyện đa phương thức của Microsoft [3], [10]. LAYOUTXLM [11] là phiên bản đa ngôn ngữ dựa trên XLM-RoBERTa, là thành phần chính trong pipeline dựa trên OCR tiếng Việt của đồ án.
- **TrOCR:** Mô hình nhận dạng chữ viết (Text Recognition) dựa hoàn toàn trên kiến trúc Transformer Encoder-Decoder [6]. TrOCR nhận đầu vào là ảnh dòng chữ được cắt từ bộ phát hiện chữ và giải mã thành văn bản, đạt hiệu năng cao nhờ cơ chế tự chú ý.

Bảng 2.1: So sánh đặc tính và giả thuyết kiểm chứng của ba hướng tiếp cận.

Phương pháp	Đặc trưng kỹ thuật	Ưu điểm & Hạn chế	Giả thuyết kiểm chứng
Baseline (Regex)	Đối sánh từ khóa và biểu thức chính quy (Regex) trên kết quả OCR.	Tốc độ xử lý cực nhanh, không cần dữ liệu huấn luyện. Tuy nhiên, luật heuristics cứng nhắc, dễ thất bại trước bố cục đa dạng.	Hoạt động hiệu quả trên các trường có định dạng cấu trúc cố định (date , total); kém hiệu quả trên các trường tự do (store_name , address).
LAYOUTXMLM (OCR-based)	Phân loại token (BIO tagging) đa phương thức: Text + Layout (2D BBox) + Visual.	Kết hợp văn bản, vị trí và ảnh trong cùng encoder. Tuy nhiên, pipeline phức tạp, phụ thuộc trực tiếp vào chất lượng và thứ tự đọc của OCR đầu vào.	Có khả năng khai thác bố cục 2D, nhưng dễ bị đứt gãy thực thể đa dòng và phụ thuộc vào độ phủ của nhãn hộp bao.
DONUT (OCR-free)	Mô hình sinh (generative) end-to-end từ ảnh sang văn bản có cấu trúc (thẻ XML).	Loại bỏ bộ OCR độc lập và tránh truyền lỗi từ OCR rời rạc. Tuy nhiên, giải mã tự hồi quy có thể sinh chuỗi sai cấu trúc, lặp hoặc không có căn cứ trong ảnh.	Cần kiểm tra mức độ ổn định của chuỗi sinh và khả năng học từ tập dữ liệu tiếng Việt quy mô hạn chế.

2.3. Nhận dạng ký tự quang học (OCR)

Hệ thống OCR thông dụng thường gồm hai giai đoạn: phát hiện văn bản và nhận dạng chữ viết. Trong pipeline dựa trên OCR của đồ án, module OCR có nhiệm vụ trích xuất văn bản thô cùng tọa độ 2D từ ảnh biên lai làm đầu vào cho LAYOUTXLM.

2.3.1. Phát hiện văn bản với PaddleOCR

Pipeline dùng module phát hiện văn bản của PADDLEOCR [2]; phần nhận dạng tích hợp của PaddleOCR bị tắt. Họ PP-OCR sử dụng bộ phát hiện dựa trên Differentiable Binarization (DB), trong đó bước nhị phân hóa được đưa vào mạng dưới dạng khả vi:

$$\hat{B}_{i,j} = \frac{1}{1 + \exp(-k(P_{i,j} - T_{i,j}))} \quad (2.1)$$

trong đó $P_{i,j}$ là bản đồ xác suất chứa ký tự, $T_{i,j}$ là bản đồ ngưỡng thích ứng học được từ backbone và FPN, và k là hệ số phóng đại khả vi. Nhờ cơ chế này, ranh giới hộp bao văn bản được tối ưu trực tiếp qua lan truyền ngược, cho độ chính xác tốt với các dòng chữ nhỏ và nghiêng trên biên lai.

2.3.2. Nhận dạng văn bản tiếng Việt với VietOCR

Ảnh các dòng chữ sau khi phát hiện được đưa vào VIETOCR [8]. Cấu hình `vgg_transformer` dùng mạng CNN họ VGG trích xuất đặc trưng thị giác, sau đó biến đổi bản đồ đặc trưng thành chuỗi để Transformer encoder-decoder sinh văn bản Unicode tiếng Việt theo cơ chế tự hồi quy.

Hàm mất mát của VietOCR là cross-entropy theo chuỗi. Với chuỗi đích $y = (y_1, y_2, \dots, y_T)$, mô hình tối ưu xác suất sinh ký tự tiếp theo dựa trên đặc trưng ảnh và các ký tự đã sinh trước đó:

$$\mathcal{L}_{\text{VietOCR}} = - \sum_{t=1}^T \log P(y_t \mid y_{<t}, \mathbf{x}). \quad (2.2)$$

Do đó VietOCR trong đồ án không dùng hàm mất mát CTC. Điểm này quan trọng khi phân tích lỗi: kết quả nhận dạng phụ thuộc vào giải mã tuần tự chứ không chỉ vào căn chỉnh đơn điệu giữa đặc trưng ảnh và ký tự.

2.4. Mô hình Donut (OCR-free)

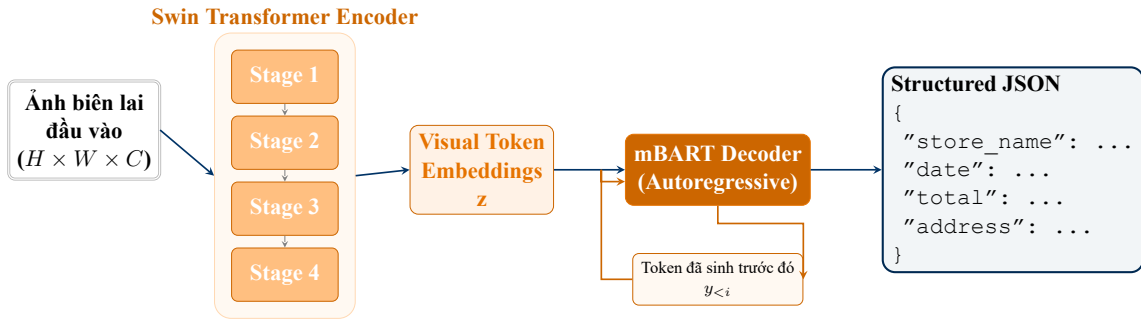
DONUT (Document Understanding Transformer) [4] là kiến trúc Transformer encoder-decoder ánh xạ trực tiếp điểm ảnh tài liệu thành chuỗi có cấu trúc mà không cần module OCR trung gian. Trong đồ án, cấu trúc đích dùng các token mở/đóng tương tự XML.

2.4.1. Kiến trúc tổng quan

Donut gồm hai thành phần chính, minh họa trong [Hình 2.1](#):

1. **Vision Encoder (Swin Transformer)**: Mã hóa ảnh biên lai thành chuỗi biểu diễn đặc trưng thị giác phân cấp $\mathbf{z} = \{z_1, z_2, \dots, z_n\}$. Swin Transformer [7] dùng cơ chế cửa sổ tự chú ý trượt (Shifted Window MSA) để giảm độ phức tạp từ bậc hai xuống tuyến tính theo kích thước ảnh, đồng thời giữ được tính liên tục ngữ cảnh.
2. **Text Decoder (mBART)**: Checkpoint donut-base dùng decoder họ mBART, một biến thể đa ngôn ngữ của BART [5]. Tại mỗi bước i , decoder kết hợp tự chú ý trên các token đã sinh và cross-attention lên biểu diễn thị giác $\mathbf{z}$ để dự đoán token tiếp theo:

$$\mathcal{L}_{\text{Donut}} = - \sum_{i=1}^T \log P(y_i | y_{<i}^*, \mathbf{z}) \quad (2.3)$$



Hình 2.1: Kiến trúc tổng quan mô hình DONUT: Vision Encoder (Swin Transformer) trích xuất đặc trưng thị giác $\mathbf{z}$ và Text Decoder (mBART) sinh chuỗi kết quả tự hồi quy [4].

2.4.2. Phân biệt khái niệm Warm-up trong Donut

Khi huấn luyện Donut, cần phân biệt rõ hai khái niệm “warm-up” dễ nhầm lẫn:

- **Learning Rate Warm-up (tỷ lệ *warmup_ratio* hoặc số bước *warmup_steps*)**: Đây là kỹ thuật điều tiết tốc độ học (LR scheduling). Trong số bước đầu tiên (thường khoảng 10% tổng số bước huấn luyện), tốc độ học được tăng dần tuyến tính từ 0 lên mức cực đại để tránh bùng nổ gradient khi mới bắt đầu cập nhật trọng số.
- **Pre-training/Warm-up mô hình trên tập CORD**: Đây là một nhánh pre-finetuning tùy chọn, trong đó mô hình có thể được huấn luyện thêm trên dữ liệu biên lai CORD trước khi fine-tune tiếng Việt. Nhánh này có trong mã nguồn nhưng không được dùng cho các kết quả chính của báo cáo.

2.5. Mô hình LayoutXLM (OCR-based)

LAYOUTXLM [11] là mô hình đa phương thức đa ngôn ngữ cho tài liệu có bố cục phong phú. Mô hình mở rộng kiến trúc đa phương thức của LayoutLMv2 bằng backbone văn bản XLM-

RoBERTa [1], qua đó xử lý được văn bản đa ngôn ngữ cùng thông tin bố cục và ảnh.

2.5.1. Nhúng đa phương thức (Multimodal Embedding)

LayoutXLM nhận đồng thời chuỗi token OCR, tọa độ 2D và ảnh toàn trang. Ba nguồn biểu diễn chính gồm:

1. **Nhúng văn bản (Text Embedding):** Mã hóa đặc trưng ngữ nghĩa của từ.
2. **Nhúng không gian 2D (Spatial Embedding):** Tọa độ bounding box $[x_1, y_1, x_2, y_2]$ của từ được chuẩn hóa về khoảng $[0, 1000]$ và nhúng qua các bảng nhúng tọa độ để mô hình nắm được vị trí tương đối và tuyệt đối trên ảnh:

$$\mathbf{e}_{2D}(\text{bbox}_i) = \mathbf{e}_{x_1}(x_1) + \mathbf{e}_{y_1}(y_1) + \mathbf{e}_{x_2}(x_2) + \mathbf{e}_{y_2}(y_2) + \mathbf{e}_w(w) + \mathbf{e}_h(h) \quad (2.4)$$

3. **Nhúng thị giác (Visual Embedding):** Ảnh toàn trang được đưa qua backbone ResNeXt-FPN để tạo bản đồ đặc trưng, sau đó pooling thành chuỗi visual token. Các visual token mang nhúng bố cục riêng và được đưa vào Transformer cùng chuỗi token văn bản; hệ thống không cắt riêng từng hộp từ để tạo vector ảnh cho từng token.

Với token văn bản, embedding đầu vào là tổng của nhúng từ, nhúng vị trí 1D và nhúng bố cục 2D. Với visual token, embedding thị giác được kết hợp với nhúng bố cục tương ứng. Hai chuỗi được nối lại và xử lý chung trong Transformer, cho phép trao đổi thông tin giữa văn bản và hình ảnh.

2.5.2. Spatial-aware Self-Attention

Cơ chế tự chú ý nhận biết không gian, kế thừa từ LayoutLMv2, thêm các bias vị trí 1D và 2D dựa trên quan hệ tương đối giữa bounding box của token i và token j :

$$\alpha_{ij} = \frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d}} + b^{1D}(j - i) + b_x^{2D}(x_{1,i} - x_{1,j}) + b_y^{2D}(y_{1,i} - y_{1,j}) \quad (2.5)$$

Nhờ vậy mô hình tự học được mối liên hệ giữa các từ nằm gần nhau trên ảnh tài liệu.

2.5.3. Gán nhãn BIO (BIO Tagging)

Bài toán trích xuất thông tin được đưa về dạng phân loại token (token classification). Mỗi token đầu vào được dự đoán một nhãn trong hệ thống BIO gồm 9 nhãn, mô tả chi tiết trong Bảng 2.2.

2.6. Các độ đo đánh giá và quy ước xử lý rỗng

Sau khi chuẩn hóa, các dự đoán được so sánh với nhãn ground-truth qua ba chỉ số: EM, NES, và CER.

Bảng 2.2: Hệ thống nhãn BIO cho 4 trường thông tin trích xuất.

Trường thông tin	Nhãn Bắt đầu (Begin)	Nhãn Bên trong (Inside)
Tên cửa hàng (store_name)	B-STORE_NAME	I-STORE_NAME
Ngày tháng (date)	B-DATE	I-DATE
Tổng tiền (total)	B-TOTAL	I-TOTAL
Địa chỉ (address)	B-ADDRESS	I-ADDRESS
Trường không quan tâm	O (Outside)	

2.6.1. Exact Match (EM)

EM đo tỷ lệ khớp chính xác hoàn toàn từng ký tự giữa dự đoán (*pred*) và nhãn thực (*gold*):

$$EM(pred, gold) = \begin{cases} 1.0 & \text{nếu } pred.strip() = gold.strip() \\ 0.0 & \text{ngược lại} \end{cases} \quad (2.6)$$

Quy ước giá trị rỗng: Nếu cả *pred* và *gold* đều là chuỗi rỗng sau chuẩn hóa $\rightarrow EM = 1.0$. Nếu chỉ một trong hai chuỗi rỗng $\rightarrow EM = 0.0$.

2.6.2. Normalized Edit Similarity (NES)

NES đo mức độ tương đồng chuỗi ký tự qua khoảng cách Levenshtein chuẩn hóa; chỉ số này không đo tương đồng ngữ nghĩa:

$$NES(pred, gold) = 1 - \frac{\text{Levenshtein}(pred, gold)}{\max(\text{len}(pred), \text{len}(gold))} \quad (2.7)$$

Quy ước giá trị rỗng: Nếu cả *pred* và *gold* đều rỗng $\rightarrow NES = 1.0$ (tránh chia cho 0). Nếu chỉ một trong hai rỗng $\rightarrow NES = 0.0$.

2.6.3. Character Error Rate (CER)

CER đo tỷ lệ lỗi ở cấp độ ký tự trên chuỗi nhãn thực:

$$CER(pred, gold) = \frac{\text{Levenshtein}(pred, gold)}{\text{len}(gold)} \quad (2.8)$$

Quy ước giá trị rỗng:

- Nếu cả hai cùng rỗng $\rightarrow CER = 0.0$.
- Nếu *gold* rỗng nhưng *pred* có giá trị $\rightarrow CER = 1.0$.
- Nếu *gold* có giá trị nhưng *pred* rỗng $\rightarrow CER = 1.0$.

Giải thích hiện tượng CER > 100%: CER có thể vượt 100% (hay > 1.0) khi chuỗi dự đoán dài hơn nhiều so với nhãn thực — thường do mô hình sinh thừa ký tự rác hoặc bị lặp chuỗi tự hồi quy (hallucination), khiến khoảng cách Levenshtein lớn hơn độ dài *gold*.

2.6.4. Hai chế độ tổng hợp kết quả

Để tránh trường hợp mô hình được điểm nhờ trả về rỗng đúng lúc ground-truth cũng rỗng, báo cáo dùng hai chế độ:

- **All samples:** đánh giá trên toàn bộ 234 mẫu, theo các quy ước giá trị rỗng ở trên.
- **Present-only:** với từng trường, chỉ đánh giá các mẫu có ground-truth khác rỗng. Đây là chế độ chính để phân tích năng lực trích xuất khi thông tin thực sự có mặt.

Macro EM, NES và CER đều là trung bình không trọng số của bốn trường.

3. Tập dữ liệu

Dưới đây là chi tiết về tập dữ liệu thực tế được sử dụng để xây dựng và đánh giá các mô hình trong báo cáo. Toàn bộ thông kê được lấy trực tiếp từ ba tệp `train.jsonl`, `val.jsonl` và `test.jsonl`; những số liệu liên quan đến ảnh thô hoặc các mẫu đã bị loại bỏ sẽ không được đề cập do không có artifact kiểm chứng đi kèm.

3.1. Nguồn dữ liệu

Dữ liệu gồm hai nguồn:

- **MC-OCR 2021** [12]: cung cấp ảnh, giá trị trường và hộp bao. Bốn nhãn nguồn được ánh xạ sang tên cửa hàng, địa chỉ, ngày và tổng tiền trong schema thống nhất.
- **Dữ liệu tự thu thập**: gồm ảnh biên lai tiếng Việt và nhãn JSON bốn trường. Trong `processed dataset` hiện tại, các mẫu này có `annotation_level=json_only`, không có hộp bao dùng được cho LayoutXLM.

Sau khi xử lý, tổng cộng có 1.559 mẫu: 1.120 mẫu từ MC-OCR và 439 mẫu tự thu thập.

3.2. Schema JSONL thống nhất

Bảng 3.1: Các trường chính trong schema JSONL.

Trường	Mô tả
ID và nhóm	<code>id</code> , <code>group_id</code> : định danh mẫu và nhóm ảnh liên quan.
Nguồn và cửa hàng	<code>source</code> , <code>store_group</code> : nguồn dữ liệu và nhóm cửa hàng.
Ảnh	<code>image_path</code> , <code>width</code> , <code>height</code> : đường dẫn và kích thước ảnh.
Mức annotation	<code>annotation_level</code> : <code>json_and_boxes</code> hoặc <code>json_only</code> .
Nhãn trường	<code>raw_target</code> , <code>target</code> : nhãn trước và sau chuẩn hóa.
Vùng trường	<code>field_boxes</code> , <code>field_polygons</code> : vùng không gian nếu có.
Oracle OCR	<code>oracle_ocr</code> : văn bản và hộp bao ground-truth khi có.
OCR cache	<code>ocr_cache_path</code> : đường dẫn tới cache PaddleOCR + VietOCR.

Về chuẩn hóa các trường đích: **date** được đưa về dạng YYYY-MM-DD, **total** thành chuỗi chữ số thuần, còn **store_name** và **address** thì chuẩn hóa Unicode NFC, xử lý khoảng trắng và loại bỏ tiền tố trường.

3.3. Chia dữ liệu và chống rò rỉ

Dữ liệu được chia theo tỷ lệ gần 70/15/15 với seed 42. Script chia sẽ nhóm theo **group_id**, phân tầng theo nguồn, đồng thời kiểm tra hash ảnh để hạn chế việc cùng một nội dung xuất hiện ở nhiều split.

Ba tệp JSONL này là nguồn sự thật duy nhất cho mọi thông kê train/validation/test trong toàn bộ báo cáo.

Bảng 3.2: Phân bố mẫu trong các tệp processed.

Split	MC-OCR	Tự thu thập	Tổng
Train	784	307	1.091
Validation	168	66	234
Test	168	66	234
Tổng	1.120	439	1.559

3.4. Mức annotation và dữ liệu huấn luyện

Bảng 3.3: Mức annotation theo split.

Split	json_and_boxes	json_only
Train	784	307
Validation	168	66
Test	168	66

Donut dùng toàn bộ train/validation vì không cần hộp bao. LayoutXLM chỉ lọc các mẫu `json_and_boxes`, nên thực tế chỉ dùng 784 mẫu train và 168 mẫu validation. Khi đánh giá, cả ba phương pháp đều chấm trên toàn bộ 234 mẫu test; những mẫu `json_only` vẫn có nhãn trường nên vẫn đánh giá được đầu ra JSON.

3.5. Giá trị ground-truth rỗng trên test

Bảng 3.4: Số mẫu có ground-truth rỗng trên 234 mẫu test.

Trường	Số mẫu rỗng	Tỷ lệ
store_name	10	4,27%
date	18	7,69%
total	17	7,26%
address	11	4,70%

Đây là lý do cần phải báo cáo present-only. Nếu mô hình cứ trả về rỗng cho mọi mẫu, nó vẫn đạt EM all-samples đúng bằng tỷ lệ ground-truth rỗng của trường đó — tức là con số EM sẽ bị phóng đại.

3.6. Kiểm tra hợp lệ

Các báo cáo validation hiện có cho thấy 0 lỗi schema, 0 ngày sai định dạng và 0 tổng tiền sai định dạng trên cả ba split. Tuy nhiên, kết quả này chỉ xác nhận dữ liệu processed hợp lệ về mặt cú pháp, chứ không đảm bảo nhãn đúng hoàn toàn về nội dung.

4. Tiền xử lý dữ liệu

Trước khi đưa dữ liệu vào mô hình, hình ảnh và văn bản đều trải qua các bước chuẩn hóa và tăng cường dữ liệu. Các cấu hình tiền xử lý hiện được giữ mặc định theo thiết lập hệ thống và chưa qua các thực nghiệm ablation để so sánh sâu hơn. Các bước xử lý này đều có ảnh hưởng đáng kể đến chất lượng trích xuất thông tin ở các giai đoạn sau.

4.1. Tiền xử lý ảnh (Image Preprocessing)

Chất lượng ảnh biên lai ảnh hưởng trực tiếp đến khả năng nhận dạng của bộ OCR. Pipeline hiện tại gọi profile mặc định `resize`; trên thực tế, profile này chỉ `resize` cạnh dài mà không tự động gọi `denoise` hay `CLAHE`:

- **Resize hình ảnh:**

- Đối với pipeline OCR (Baseline và LayoutXLM): Ảnh được `resize` theo cạnh dài nhất với `max_long_side = 1600` pixel để giữ nét chữ rõ ràng và không thay đổi tỷ lệ khung hình (aspect ratio).
- Đối với Donut: Ảnh được `resize` về kích thước cố định 1280×960 pixel, phần thiếu được lấp đầy bằng khoảng trắng (padding).

Tập cấu hình có chứa cờ `denoise=true` và `clahe=true`, nhưng pipeline tiền xử lý thực tế chưa đọc hay thực thi hai cờ này. Do đó báo cáo coi chúng là cấu hình dự kiến chứ chưa phải bước đã áp dụng. Ngoài ra, mã nguồn còn hỗ trợ thêm:

- **Nhị phân hóa (Binarize):** `Resize` cạnh dài rồi áp dụng adaptive Gaussian threshold, không phải Otsu.
- **Nắn thẳng ảnh (Rectify):** Tìm contour bốn góc và dùng perspective transform; nếu không tìm được contour hợp lệ thì giữ nguyên ảnh gốc.

4.2. Tăng cường dữ liệu (Data Augmentation)

Augmentation được áp dụng trực tuyến, chỉ trên tập train. Hai mô hình dùng hai nhóm phép biến đổi khác nhau do LayoutXLM cần giữ tọa độ OCR và hộp nhãn khớp với ảnh.

Bảng 4.1: Các phép tăng cường dữ liệu được cài đặt cho hai mô hình.

Mô hình	Biến đổi hình học	Biến đổi quang học
DONUT	Affine: dịch $\pm 2\%$, scale 0,97–1,03, xoay $\pm 8^\circ$; perspective 0,01–0,025. Không áp dụng phép biến đổi làm thay đổi hình học để giữ nguyên tính hợp lệ của bounding box.	Brightness/contrast, Gaussian blur, Gaussian noise và random shadow.
LAYOUTXLM		Brightness/contrast, Gaussian blur và random shadow.

4.3. Chuẩn hoá văn bản (Text Normalization)

Chuẩn hóa văn bản là bước bắt buộc để đồng nhất định dạng đầu ra. Mã nguồn không tự sửa chính tả, không bỏ dấu, cũng không chuyển toàn bộ chuỗi về chữ thường. Vì vậy EM vẫn phân biệt hoa/thường và nhạy với dấu tiếng Việt.

1. Chuẩn hóa chung (General Normalization) Áp dụng cho toàn bộ các trường thông tin:

- **Unicode NFC:** Chuyển đổi toàn bộ văn bản về dạng chuẩn tổ hợp Unicode NFC để thống nhất cách biểu diễn ký tự có dấu.
- **Whitespace:** Gộp các khoảng trắng liên tiếp và cắt khoảng trắng ở hai đầu chuỗi.
- **Ký tự đặc biệt:** Loại bỏ các ký tự điều khiển không in được.

2. Chuẩn hóa theo trường (Field-specific Normalization)

- **store_name:** Loại bỏ tiền tố cửa hàng hoặc siêu thị ở đầu chuỗi, có hỗ trợ biến thể không dấu.
- **address:** Loại bỏ tiền tố địa chỉ, địa chỉ hoặc address ở đầu chuỗi.
- **date:** Chuyển đổi nhiều định dạng ngày tự do về dạng chuẩn ISO YYYY-MM-DD. Nếu phân tích cú pháp thất bại, trả về chuỗi rỗng.
- **total:** Loại bỏ đơn vị tiền tệ ("đ", "đồng", "VND"), dấu chấm/phẩy phân cách hàng nghìn và các số 0 ở đầu.

Bảng 4.2 minh họa kết quả chuẩn hóa cho từng trường.

Bảng 4.2: Ví dụ chuẩn hoá văn bản theo trường thông tin.

Trường	Văn bản gốc trước chuẩn hóa	Văn bản sau chuẩn hóa
store_name	Cửa hàng CIRCLE K	CIRCLE K
address	Địa chỉ: 123 Nguyễn Trãi, Thanh Xuân, HN	123 Nguyễn Trãi, Thanh Xuân, HN
date	Lúc 15/03/2021 14:30:15	2021-03-15
total	Tổng cộng: 115.000 đ	115000

4.4. Xây dựng OCR Cache

Việc chạy OCR (PaddleOCR kết hợp VietOCR) tốn khá nhiều tài nguyên và thời gian. Để không phải chạy lại OCR mỗi lần thí nghiệm, hệ thống xây dựng OCR cache — lưu lại kết quả nhận dạng của từng ảnh để dùng lại ngay trong các tác vụ học máy tiếp theo.

OCR cache chứa danh sách từ, bounding box, kích thước ảnh sau tiền xử lý và metadata phiên bản OCR. Cache này dùng chung cho cả Baseline và LayoutXLM, tránh phải chạy lại OCR khi huấn luyện, suy luận hay phân tích lỗi. Báo cáo không công bố thời gian OCR CPU/GPU vì artifact hiện tại không có log đo đầy đủ để xác minh.

4.5. Trạng thái khảo sát tiền xử lý OCR

Script khảo sát hỗ trợ bốn profile `none`, `resize`, `binarize` và `rectify`, dự kiến chạy trên `validation` thay vì `test` để tránh chọn cấu hình dựa trên tập kiểm thử. Tuy nhiên, các CSV lưu trong repository chỉ chứa ba mẫu mỗi profile và không tái lập được bảng 100 mẫu từng có trong bản thảo trước. Vì vậy, báo cáo bỏ bảng xếp hạng ablation và không kết luận profile `resize` là tối ưu.

Profile `resize` vẫn được giữ làm mặc định vì đây là lựa chọn đã được tích hợp nhất quán trong OCR cache và inference. Nếu muốn làm ablation hợp lệ trong tương lai, cần cố định danh sách mẫu validation, đảm bảo đủ cache cho từng profile, báo cáo cả độ phủ cache và không cộng điểm tự động cho trường ground-truth rỗng.

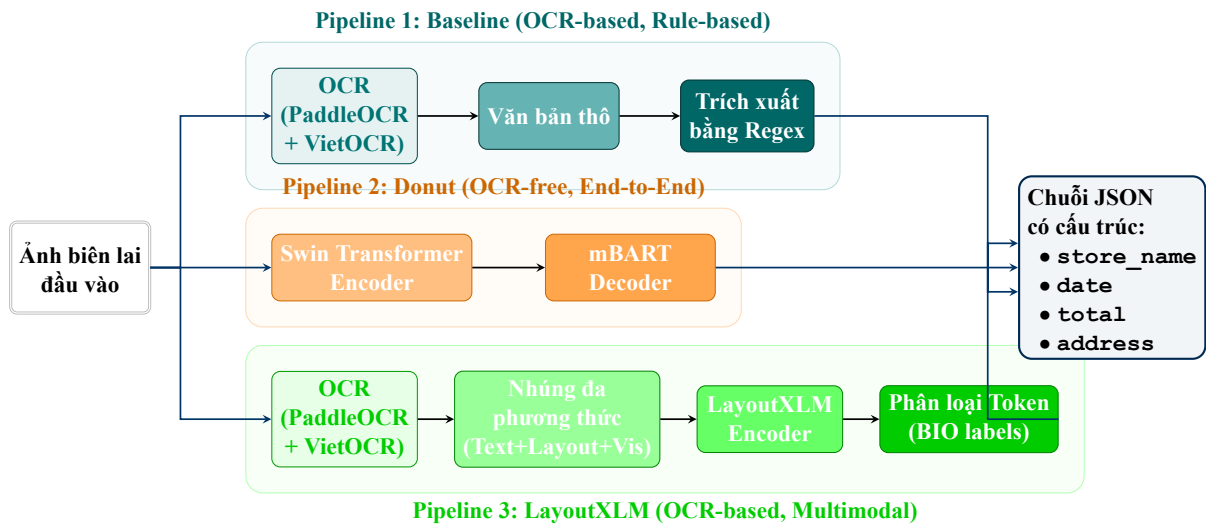
5. Phương pháp thực hiện

Báo cáo so sánh ba hướng tiếp cận (pipeline) khác nhau cho bài toán trích xuất thông tin biên lai tiếng Việt: phương pháp Baseline dựa trên luật, mô hình sinh end-to-end Donut, và mô hình đa phương thức LayoutXLM kết hợp với VietOCR. Nội dung tiếp theo trình bày chi tiết thiết kế và quy trình hoạt động của từng pipeline.

5.1. Tổng quan kiến trúc hệ thống

Hệ thống được cấu trúc theo dạng các module độc lập, cho phép kiểm thử và đánh giá từng kiến trúc một cách riêng biệt. Sơ đồ tổng quan của các pipeline được thể hiện trong [Hình 5.1](#). Đầu vào là ảnh biên lai, đầu ra là tệp JSON chứa bốn trường: **store_name**, **date**, **total**, **address**.

- **Pipeline 1 — Baseline (Rule-based):** Ảnh → Optical Character Recognition — Nhận dạng ký tự quang học (OCR) (PaddleOCR + VietOCR) → Luật / Regex → JSON.
- **Pipeline 2 — DONUT (OCR-free):** Ảnh → Swin Encoder → mBART Decoder → JSON.
- **Pipeline 3 — VietOCR + LAYOUTXLM (OCR-based):** Ảnh → OCR → Token + BBox → LAYOUTXLM → BIO → JSON.



Hình 5.1: Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.

5.2. Baseline: OCR + Trích xuất dựa trên luật (Rule-based)

Baseline hoạt động như một hệ chuyên gia đơn giản, dựa trên giả định rằng các trường thông tin quan trọng thường nằm gần các từ khóa đặc trưng hoặc tuân theo một quy luật nhất định trên biên lai. Phương pháp dùng các quy tắc thủ công (heuristic) và biểu thức chính quy (regular expression — regex) để trích xuất thông tin từ kết quả OCR. Đây là phương pháp đơn giản nhất, không cần huấn luyện mô hình học sâu, nhưng phụ thuộc nhiều vào chất lượng OCR và sự nhất quán của bố cục biên lai.

5.2.1. Pipeline tổng quan

Quy trình xử lý gồm hai bước chính:

1. **Nhận dạng văn bản:** Ảnh biên lai được đưa qua bộ phát hiện vùng chữ PaddleOCR [2] và bộ nhận dạng VietOCR [8] để thu được danh sách các cặp (văn bản, bounding box) sắp xếp theo thứ tự đọc (reading order).
2. **Trích xuất theo luật:** Áp dụng các quy tắc heuristic và regex riêng cho từng trường thông tin để trích xuất giá trị từ danh sách văn bản.

5.2.2. Quy tắc trích xuất cụ thể

Cách trích xuất cho từng trường như sau:

- **Tên cửa hàng (store_name):** Chấm điểm tối đa 5 dòng đầu theo vị trí, tỷ lệ chữ hoa và độ dài. Các dòng chứa từ khóa hóa đơn, mã số thuế, điện thoại hoặc địa chỉ, dòng quá ngắn và dòng có tỷ lệ chữ số cao bị loại.
- **Ngày tháng (date):** Duyệt từ trên xuống, chuẩn hóa từng dòng và lấy ngày hợp lệ đầu tiên trong khoảng năm 1990–2040 theo hằng số `VALID_YEAR_RANGE`. Bộ chuẩn hóa hỗ trợ dấu /, -, . và năm hai chữ số.
- **Tổng tiền (total):** Duyệt từ cuối hóa đơn lên để ưu tiên từ khóa tổng tiền gần phần thanh toán. Với mỗi dòng có từ khóa, xét dòng đó và tối đa một dòng kế tiếp, lọc số điện thoại, mã số thuế, ngày và giá trị dưới 1.000 rồi chọn số tiền lớn nhất. Nếu không có từ khóa, thuật toán tìm ứng viên hợp lệ trong một phần ba cuối hóa đơn.
- **Địa chỉ (address):** Tìm dòng có từ khóa địa chỉ nhưng không chứa ngày, điện thoại, mã số thuế hoặc từ khóa thanh toán. Nếu các dòng kế tiếp không bị chặn và có nội dung đủ dài, nối thêm tối đa ba dòng kế tiếp trước khi chuẩn hóa.

Các quy tắc trên được xây dựng dựa trên quan sát thực tế trên tập dữ liệu biên lai tiếng Việt. [Bảng 5.1](#) tóm tắt các quy tắc chính của từng trường.

Bảng 5.1: Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.

Trường	Mô tả quy tắc
store_name	Chấm điểm tối đa 5 dòng đầu; loại các dòng giống tiêu đề hóa đơn, mã số thuế, điện thoại, địa chỉ hoặc chứa quá nhiều chữ số.
date	Tìm chuỗi khớp regex ngày tháng, bỏ qua các từ khoá: ngày, date, time, và chỉ giữ năm trong khoảng 1990–2040.
total	Duyệt từ cuối lên, xét dòng có từ khóa và một dòng kế tiếp; nếu không có từ khóa thì tìm số tiền hợp lệ trong một phần ba cuối.
address	Tìm dòng có từ khóa địa chỉ, loại ngữ cảnh bị chặn và nối tối đa ba dòng kế tiếp hợp lệ.

Các mẫu regex dùng cho trường **date** và **total** được trình bày trong [Đoạn mã 1](#).

```

1 # --- Regex cho truong date ---
2 DATE_PATTERNS = [
3     r'(\d{1,2})[/-](\d{1,2})[/-](\d{4})',    # dd/mm/yyyy hoac dd-mm-
        yyyy
4     r'(\d{1,2})[/-](\d{1,2})[/-](\d{2})',    # dd/mm/yy
5     r'(\d{4})[/-](\d{1,2})[/-](\d{1,2})',    # yyyy/mm/dd (ISO)
6 ]
7 DATE_IGNORE_KW = ['ngay', 'date', 'time']
8
9 # --- Regex cho truong total ---
10 TOTAL_KEYWORDS = ['tong cong', 'thanh toan', 'total', 'phai tra']
11 MONEY_PATTERN = r'\d[\d\.\s]*'    # so tien: 150.000, 1,500,000
12 TOTAL_SEARCH_RANGE = 1            # dong keyword va mot dong ke
        tiep

```

Đoạn mã 1: Các mẫu regex dùng trong pipeline baseline.

5.3. Donut: End-to-End OCR-free

Donut được chọn để kiểm tra xem liệu một mạng nơ-ron có thể tự học cách “đọc” và “hiểu” bố cục biên lai trực tiếp từ pixel, không cần bước OCR riêng. Nếu làm được, hệ thống sẽ không còn phụ thuộc vào chất lượng bộ OCR ngoài. Chi tiết kiến trúc Swin Encoder + mBART Decoder đã được trình bày ở [Mục 2.4](#).

5.3.1. Chuẩn bị dữ liệu

Chuỗi đích dùng cấu trúc thẻ kiểu XML để biểu diễn nhãn và quan hệ phân cấp của từng trường. Với cách này, BART Decoder có thể học sinh ra các thẻ đóng/mở cùng nội dung văn bản tương ứng mà không cần thêm cấu trúc biểu diễn phức tạp. Mỗi mẫu huấn luyện là một cặp (ảnh biên lai, chuỗi đích). Chuỗi đích được định dạng bằng các token đặc biệt đánh dấu từng trường thông tin. Ảnh đầu vào được resize về kích thước 1280×960 pixel, giữ nguyên tỷ lệ khung hình (aspect ratio) bằng cách thêm padding.

Ví dụ chuỗi đích cho một biên lai:

```

1 <s_receipt_ie>
2   <s_store_name>CUA HANG TIEN LOI GS25</s_store_name>
3   <s_date>15/03/2024</s_date>
4   <s_total>125.000</s_total>
5   <s_address>123 Nguyen Trai, Thanh Xuan, Ha Noi</s_address>
6 </s_receipt_ie>

```

Đoạn mã 2: Ví dụ chuỗi đích (target string) cho mô hình DONUT.

5.3.2. Cấu hình huấn luyện

Cấu hình siêu tham số huấn luyện được thiết lập tương thích với tài nguyên GPU Tesla T4 16GB. Cụ thể, batch size thực tế được đặt là 2 kết hợp với tích lũy gradient 8 bước nhằm đạt batch size hiệu dụng bằng 16. Thiết lập này chủ yếu dựa trên giới hạn phần cứng sẵn có và chưa trải qua các lượt tối ưu hóa tham số nâng cao. [Bảng 5.2](#) liệt kê cấu hình huấn luyện của mô hình DONUT.

Bảng 5.2: Cấu hình huấn luyện mô hình DONUT.

Tham số	Giá trị
Pretrained model	naver-clova-ix/donut-base
Max length	768 tokens
Epochs	150
Batch size	2
Gradient accumulation	8 (effective batch size = 16)
Learning rate	2×10^{-5}
LR scheduler	Cosine
Warmup ratio	0.1
FP16	Có
Gradient checkpointing	Có
Early stopping patience	15 epochs

5.3.3. Suy luận (Inference)

Khi suy luận, Donut nhận ảnh biên lai cùng prompt `<s_receipt_ie>`, sau đó decoder sinh tuần tự các token tiếp theo. Bộ parser tìm nội dung nằm giữa các cặp token mở/đóng; trường nào không tạo được cặp token hợp lệ sẽ được gán chuỗi rỗng trước khi chuẩn hóa. Cụ thể, mô hình DONUT nhận ảnh biên lai làm đầu vào và sinh chuỗi văn bản qua beam search với `num_beams=2`. Chuỗi đầu ra được parse để lấy giá trị của từng trường rồi chuyển thành định dạng JSON.

Quy trình suy luận:

1. Ảnh biên lai được resize về 1280×960 và đưa qua encoder.
2. Decoder sinh chuỗi đầu ra sử dụng beam search (`num_beams=2`).
3. Chuỗi đầu ra được parse bằng regex để trích xuất nội dung giữa các cặp token `<s_field>` và `</s_field>`.
4. Kết quả được ghi vào tệp JSON.

Trạng thái artifact Donut: Model Collapse

Lượt prediction được dùng trong báo cáo hiện tại truyền `generation_max_length=768` vào hàm `generate()`, khắc phục lỗi giới hạn 20 token mặc định. Tuy nhiên, kết quả dự đoán của Donut (Macro EM 0%) cho thấy hiện tượng sụp đổ mô hình (model collapse): mô hình liên tục sinh lặp từ hoặc sinh ra các cặp thẻ XML không hợp lệ. Do đó, kết quả này được báo cáo như một ca thất bại thực nghiệm (negative result) của checkpoint hiện tại, chứ không phải là đại diện tiêu chuẩn để đánh giá hiệu năng của kiến trúc OCR-free.

5.4. VietOCR + LayoutXLM: OCR-based Layout-aware

Hướng tiếp cận này kết hợp VietOCR và LAYOUTXLM: VietOCR nhận dạng chữ viết tiếng Việt có dấu, LayoutXLM tích hợp văn bản với thông tin không gian (tọa độ bounding box) và ảnh để phân loại thực thể theo lược đồ BIO. Chi tiết kiến trúc đa phương thức của LayoutXLM đã được trình bày ở Mục 2.5.

5.4.1. Chuẩn bị dữ liệu

LayoutXLM chỉ dùng các mẫu có `annotation_level=json_and_boxes`. Với mỗi từ OCR, mã nguồn tính tỷ lệ phần diện tích của hộp từ nằm trong hộp ground-truth:

$$\text{coverage}(W, F) = \frac{|W \cap F|}{|W|}. \quad (5.1)$$

Đây là tỷ lệ bao phủ hộp từ, không phải Intersection over Union. Từ được gán vào trường có coverage cao nhất nếu tỷ lệ đạt ít nhất 0,5; các từ còn lại nhận nhãn O. **Đầu vào:** Kết quả OCR từ cache bao gồm văn bản và bounding box cho từng từ (word-level).

Gán nhãn BIO: Các từ liên tiếp cùng thuộc một trường nhận lần lượt nhãn B-FIELD và I-FIELD. Khi một từ không đạt ngưỡng coverage, chuỗi thực thể hiện tại kết thúc.

Tokenization: Văn bản được tokenize với giới hạn 512 token. Tham số `doc_stride` được đặt là 128 trong cấu hình, nhưng dataset và inference hiện chưa tạo sliding windows. Chuỗi dài bị cắt ở giới hạn token, nên các trường nằm cuối biên lai có thể bị mất.

5.4.2. Cấu hình huấn luyện

Siêu tham số lấy từ cấu hình thực nghiệm trong repository. Số epoch tối đa là 20 và early stopping patience là 5. Bảng dưới đây chỉ mô tả cấu hình khai báo; bảng chứng về số bước và epoch thực chạy được tổng hợp riêng trong Mục 6.6. Bảng 5.4 liệt kê cấu hình huấn luyện của mô hình LAYOUTXLM.

Hệ thống nhãn BIO gồm 9 nhãn được mô tả chi tiết trong Bảng 2.2 ở Mục 2.

Bảng 5.3: Ví dụ tối giản về gán nhãn **BIO** cho một chuỗi OCR.

Từ OCR	Nhãn BIO
Co.opmart	B-STORE_NAME
Đồng	I-STORE_NAME
Đa	I-STORE_NAME
Ngày:	O
16/05/2024	B-DATE
TỔNG CỘNG	O
245.000	B-TOTAL
Địa chỉ:	O
Hà	B-ADDRESS
Nội	I-ADDRESS

Bảng 5.4: Cấu hình huấn luyện mô hình LAYOUTXLM.

Tham số	Giá trị
Pretrained model	microsoft/layoutxlm-base
Max length	512 tokens
Doc stride	128 (đã khai báo, chưa dùng sliding window)
Số nhãn (labels)	9 nhãn BIO
Epochs	20
Batch size	8
Gradient accumulation	4 (effective batch size = 32)
Learning rate	2×10^{-5}
LR scheduler	Linear
Warmup ratio	0.1
FP16	Có
Early stopping patience	5 epochs

Tập train LayoutXLM có 784 mẫu và validation có 168 mẫu có hộp bao. Các mẫu `json_only` không tham gia được vào huấn luyện token classification. Trong khi đó, Donut không cần hộp bao nên dùng toàn bộ 1.091 mẫu train và 234 mẫu validation.

5.4.3. Suy luận và hậu xử lý (Inference & Post-processing)

Thuật toán gộp nhãn hoạt động theo kiểu máy trạng thái: `B-FIELD` mở span mới, `I-FIELD` cùng loại thì nối vào span đang mở. Nếu gặp `I-FIELD` mà không có span cùng loại đang mở, triển khai hiện tại vẫn mở span mới để không mất thực thể — tương đương chính sách sửa chuỗi BIO bị đứt. Nhãn `O` hoặc nhãn trường khác sẽ đóng span hiện tại. Quy trình suy luận của pipeline LAYOUTXLM:

1. **Dự đoán nhãn BIO:** Mô hình LAYOUTXLM nhận đầu vào gồm token, bounding box và ảnh, sau đó dự đoán nhãn BIO cho từng token.
2. **Gộp span và chọn ứng viên:** Sau khi tạo span, tên cửa hàng lấy span đầu tiên; ngày lấy span đầu tiên chuẩn hóa hợp lệ; tổng tiền loại số giống điện thoại/MST hoặc quá nhỏ rồi chọn giá trị lớn nhất; các span địa chỉ gần nhau theo trục y được ghép thành block và ưu tiên block có từ khóa địa chỉ hoặc có độ dài lớn nhất.
3. **Chuẩn hoá:** Văn bản trích xuất được chuẩn hoá (loại bỏ khoảng trắng thừa, chuẩn hoá Unicode [Unicode Normalization Form Composed](#) — [Dạng chuẩn hoá Unicode tổ hợp \(NFC\)](#)) trước khi ghi vào tệp JSON đầu ra.

5.5. Hậu xử lý chung (Post-processing)

Hậu xử lý chung có nhiệm vụ chuẩn hóa schema và một số định dạng trường trước khi đánh giá. Kết quả đầu ra của cả ba pipeline đều đi qua bước hậu xử lý này trước khi so sánh với ground-truth, để đảm bảo nhất quán khi đánh giá. Cụ thể gồm:

- **Chuẩn hoá Unicode:** Chuyển đổi toàn bộ văn bản về dạng NFC để thống nhất cách biểu diễn ký tự tiếng Việt.
- **Chuẩn hoá khoảng trắng:** Loại bỏ khoảng trắng đầu/cuối và thay thế các chuỗi khoảng trắng liên tiếp bằng một khoảng trắng đơn.
- **Định dạng thống nhất:** Đảm bảo tất cả các trường đầu ra đều có mặt trong tệp JSON, sử dụng chuỗi rỗng cho các trường không trích xuất được.

Triển khai hiện tại không chuyển `store_name` và `address` sang chữ thường; EM vì vậy phân biệt hoa/thường. `date` được parse về ISO và `total` chỉ giữ chữ số.

6. Thiết lập thực nghiệm

Dưới đây là thống kê các cấu hình lưu trữ trong repository, phạm vi dữ liệu sử dụng của từng mô hình và quy trình khởi tạo các tập kết quả dự đoán. Nhóm nghiên cứu cam kết chỉ trình bày thông tin có log huấn luyện gốc (các số liệu không có bằng chứng đối chiếu như thời gian chạy thực tế hay epoch dừng thực tế sẽ được bỏ qua nhằm đảm bảo tính minh bạch).

6.1. Trạng thái tập kết quả dự đoán

Bảng 6.1: Trạng thái bằng chứng của các prediction artifact đã khóa bằng SHA-256.

Artifact	Trạng thái	n	SHA-256	Phạm vi sử dụng
Baseline	Hợp lệ	234	a16c4d903e...	Bảng chính, RQ1, latency thành phần
VietOCR + LAYOUTXLM	Hợp lệ, kèm sensitivity check	234	c65ffdca7f...	Bảng chính, RQ1, latency thành phần
DONUT	Hợp lệ, diễn giải như model collapse	234	1550902cb1...	Bảng chính, RQ1, phụ lục chẩn đoán

Baseline, LayoutXLM và Donut đều có đủ 234 prediction, không có bản ghi `status=error` và được đưa vào bảng định lượng chính. Với LayoutXLM, bốn prediction cũ dùng hộp OCR trên ảnh resize nhưng chuẩn hóa tọa độ theo kích thước ảnh thô; báo cáo giữ nguyên artifact và bổ sung sensitivity check loại bốn ID này. Với Donut, checkpoint config không lưu `max_length`, nên manifest ghi fallback metadata 20 token nếu không override; lượt prediction hiện tại truyền `explicit generation_max_length=768` vào `generate()`. Kết quả Donut trong đồ án được xem là một ca thất bại thực nghiệm của checkpoint hiện tại. Nhóm không dùng kết quả này để kết luận rằng hướng OCR-free kém hơn OCR-based trong mọi điều kiện. SHA-256 trong bảng khóa đúng nội dung các artifact được phân tích.

6.2. Môi trường và cấu hình

Các checkpoint được huấn luyện trên GPU NVIDIA Tesla T4. Về phần mềm, dự án dùng Python, PyTorch, Hugging Face Transformers, PaddleOCR, VietOCR và RapidFuzz. Hạt giống chia dữ liệu là 42. Do repository không lưu đầy đủ ảnh chụp môi trường cùng log package từ lần huấn luyện tạo checkpoint, các phiên bản thư viện ghi trong tài liệu hướng dẫn chỉ nên xem là cấu hình dự kiến để tái lập, không phải bằng chứng về môi trường chạy trước đó.

Các giá trị trong [Bảng 6.2](#) lấy từ file cấu hình hiện tại. Chúng không cho biết số epoch thực chạy, checkpoint tốt nhất hay thời gian huấn luyện.

Bảng 6.2: Cấu hình huấn luyện được khai báo cho Donut và LayoutXLM.

Siêu tham số	DONUT	LAYOUTXLM
Mô hình nền tảng	donut-base	layoutxlm-base
Epoch tối đa	150	20
Batch size mỗi GPU	2	8
Gradient accumulation	8	4
Learning rate	2×10^{-5}	2×10^{-5}
LR scheduler	Cosine	Linear
Warm-up ratio	0,10	0,10
Mixed precision	FP16	FP16
Early stopping patience	15	5

6.3. Phạm vi dữ liệu của từng pipeline

Bảng 6.3: Số mẫu có thể sử dụng ở từng giai đoạn.

Phương pháp	Train	Validation	Test
Baseline	Không huấn luyện	Không áp dụng	234
DONUT	1.091	234	234
LAYOUTXLM	784	168	234

Donut chỉ cần ảnh và nhãn JSON nên dùng được cả hai nguồn dữ liệu. LayoutXLM cần hộp bao trường để sinh nhãn BIO, do đó chỉ huấn luyện trên các mẫu `json_and_boxes`, mất 307 mẫu (28% tập train) so với Donut. Cả ba phương pháp đều chấm trên cùng 234 ID test. Tuy nhiên, với Macro EM chênh lệch chỉ 2,55 điểm giữa LayoutXLM và Baseline, mức chênh dữ liệu huấn luyện 28% này đủ sức làm đảo chiều kết quả. Do đó, mọi phép đối chiếu trong báo cáo là **so sánh giữa các artifact trong điều kiện đồ án**, không phải so sánh kiến trúc thuần túy trong điều kiện dữ liệu đồng nhất.

6.4. Quy trình suy luận và đánh giá

Predictions của ba phương pháp được lưu theo cùng schema, bao gồm kết quả thô, kết quả chuẩn hóa, trạng thái xử lý và các thành phần latency. Evaluator ghép prediction với ground-truth theo **id**. Prediction thiếu hoặc có `status=error` sẽ được chấm như dự đoán rỗng thay vì bị loại khỏi mẫu số.

Kết quả được báo cáo theo hai chế độ:

1. **All samples:** toàn bộ 234 mẫu; trường hợp cả prediction và ground-truth rỗng được tính đúng theo quy ước metric.
2. **Present-only:** chỉ các mẫu có ground-truth khác rỗng của từng trường. Chế độ này được ưu tiên khi phân tích khả năng trích xuất thực tế.

Các tệp JSON metrics, CSV phân tích lỗi, biểu đồ và bảng LaTeX đều được sinh lại từ predictions hiện có. Bảng trong chương kết quả được nạp từ thư mục `report/generated` để tránh sao chép số liệu bằng tay.

6.5. Phạm vi phép đo latency

Artifact prediction chỉ cho phép rút ra kết luận về thành phần thời gian đã được ghi:

- **Baseline:** thời gian áp dụng luật và hậu xử lý trên OCR cache.
- **LayoutXLM:** thời gian suy luận mô hình từ OCR cache.
- **Donut:** thời gian suy luận trực tiếp từ ảnh; phép đo bị ảnh hưởng bởi sinh tự hồi quy dài/lặp của checkpoint hiện tại.

Thời gian chạy PaddleOCR và VietOCR từ ảnh thô không được ghi đồng nhất trong các prediction hiện có. Vì vậy, Baseline và LayoutXLM có thể so sánh trực tiếp với nhau ở phạm vi sau OCR cache, còn Donut được báo cáo riêng trong cùng bảng như thời gian suy luận trực tiếp từ ảnh. Báo cáo không cộng thêm một hằng số OCR giả định và không gọi các phép đo này là so sánh end-to-end đồng nhất từ ảnh thô.

6.6. Trạng thái bằng chứng huấn luyện

Các thực nghiệm chạy lại trên Kaggle đã ghi nhận đầy đủ tệp `trainer_state.json` của quá trình huấn luyện:

- **LayoutXLM (ocr_cache):** Tiến trình dừng tại bước 200 (tương đương 15,4 epochs) với F1 đạt 83,65% trên tập validation.
- **LayoutXLM (oracle_ocr):** Chạy đầy đủ 80 bước (20 epochs) trên tập dữ liệu MC-OCR rút gọn có ground-truth OCR (256 mẫu train).

Các log huấn luyện chi tiết này hiện được lưu trong thư mục `outputs/training_logs/` làm bằng chứng kiểm chứng và tái lập kết quả.

Thiếu đường cong huấn luyện

Do cấu hình `eval_steps=200` và `logging_steps=20`, tệp `trainer_state.json` của LayoutXLM chỉ ghi 10 điểm training loss (mỗi 20 bước) và duy nhất 1 lần đánh giá trên tập validation tại bước 200. Training loss giảm đều từ 1,29 (epoch 1,6) xuống 0,07 (epoch 15,4), cho thấy tín hiệu hội tụ trên tập train. Tuy nhiên, chỉ có 1 điểm eval F1 = 83,65% nên không thể vẽ đường cong validation hoàn chỉnh hay đánh giá liệu mô hình có overfit trong các epoch cuối. Đây là hạn chế do `eval_steps` quá thưa so với tổng 260 bước huấn luyện; các lần chạy sau cần giảm `eval_steps` để thu thập nhiều điểm đánh giá hơn.

Tiêu chí checkpoint và artifact đã đóng băng

Mã huấn luyện LAYOUTXLM hiện tại đã được chỉnh để các lần huấn luyện sau chọn checkpoint tốt nhất theo entity-level F1 từ `segeval`; fallback token-accuracy chỉ được phép bật bằng `-allow_metric_fallback` cho smoke test. Tuy nhiên, các bảng định lượng trong báo cáo vẫn lấy từ prediction artifact đã đóng băng và không rerun training trong đợt chốt này. Nếu artifact lịch sử được sinh từ checkpoint chọn theo loss, kết quả cần được diễn giải theo checkpoint/setting lịch sử đó thay vì ngầm hiểu là đã được sinh từ checkpoint chọn theo F1.

7. Kết quả thực nghiệm và đánh giá

Kết quả dưới đây được tính lại trực tiếp từ các prediction artifact đóng băng trên 234 mẫu test. Các bảng metrics và latency được sinh tự động bởi script `scripts/generate_report_artifacts.py` nhằm đảm bảo tính đồng nhất giữa báo cáo và các tệp JSON kết quả. Cả ba phương pháp Baseline, LAYOUTXLM và DONUT đều có mặt trong bảng định lượng; riêng Donut được phân tích cụ thể dưới dạng một ca chẩn đoán lỗi hội tụ (model collapse) của checkpoint hiện có.

7.1. Kết quả trên toàn bộ mẫu và trên trường có mặt

Bảng 7.1: Kết quả trên toàn bộ 234 mẫu kiểm thử của các artifact hợp lệ. Macro là trung bình không trọng số của bốn trường.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	Macro
Exact Match (EM %) ↑					
Baseline	25,21	74,79	54,27	4,70	39,74
VietOCR + LAYOUTXLM	29,06	61,11	71,37	13,68	43,80
DONUT	4,27	7,69	7,26	4,70	5,98
Normalized Edit Similarity (NES %) ↑					
Baseline	40,84	77,01	67,86	30,29	54,00
VietOCR + LAYOUTXLM	56,78	62,18	76,11	59,88	63,74
DONUT	4,57	7,69	7,26	4,70	6,05
Character Error Rate (CER %) ↓					
Baseline	91,35	22,99	37,00	79,49	57,71
VietOCR + LAYOUTXLM	49,51	37,82	23,99	43,98	38,82
DONUT	993,21	92,31	92,74	95,30	318,39

Bảng 7.1 phản ánh hành vi của các artifact trên toàn bộ test set, bao gồm trường hợp trường thông tin không có nhãn. Bảng 7.2 loại các ground-truth rỗng theo từng trường và được dùng làm cơ sở chính để đánh giá khả năng trích xuất. Bảng 7.3 bổ sung tỷ lệ prediction khác rỗng để phân biệt chất lượng với xu hướng trả về rỗng.

Trên chế độ present-only, LAYOUTXLM đạt Macro EM 42,96% và Macro NES 63,89%, cao hơn Baseline tương ứng 40,41% và 55,47%. Khoảng cách Macro EM chỉ là 2,55 điểm phần trăm, nên kết luận phù hợp không phải LayoutXLM áp đảo Baseline, mà là LayoutXLM cải thiện ổn định hơn ở các trường cần ngữ cảnh/bổ cục như tổng tiền và địa chỉ. Cần lưu ý rằng LayoutXLM chỉ được huấn luyện trên 784 mẫu (ít hơn 28% so với Donut), nên đây là so sánh artifact trong điều kiện đồ án chứ không phải so sánh kiến trúc thuần túy. Nếu mục tiêu là triển khai thực tế với tài nguyên hạn chế, mức tăng Macro EM 2,55 điểm cần được cân nhắc cùng chi phí vận hành của pipeline LAYOUTXLM, gồm GPU, OCR cache, xử lý bbox và bảo trì checkpoint. DONUT đạt Macro EM 0,00% và CER lên đến 334% (all-samples) hay 1.037% (present-only) trong thiết lập

Bảng 7.2: Kết quả present-only của các artifact hợp lệ: mỗi trường chỉ được chấm trên mẫu có ground-truth khác rỗng.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	Macro
Exact Match (EM %) ↑					
Baseline	26,34	75,46	58,06	1,79	40,41
VietOCR + LAYOUTXLM	28,12	60,19	71,89	11,66	42,96
DONUT	0,00	0,00	0,00	0,00	0,00
Normalized Edit Similarity (NES %) ↑					
Baseline	42,66	77,87	72,72	28,64	55,47
VietOCR + LAYOUTXLM	57,08	61,34	77,01	60,14	63,89
DONUT	0,31	0,00	0,00	0,00	0,08
Character Error Rate (CER %) ↓					
Baseline	90,97	22,13	32,53	81,62	56,81
VietOCR + LAYOUTXLM	49,49	38,66	23,10	43,90	38,79
DONUT	1037,55	100,00	100,00	100,00	334,39

Bảng 7.3: Độ phủ prediction và số mẫu đánh giá. Số mẫu present-only lần lượt là 224, 216, 217 và 223 cho bốn trường.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	n	Lỗi suy luận
Baseline	98,29	74,79	91,88	61,54	234	0
VietOCR + LAYOUTXLM	73,93	58,97	76,50	75,64	234	0
DONUT	9,40	0,00	0,00	0,00	234	0

hiện tại. Các giá trị CER vượt quá 100% là hợp lệ về mặt toán học: CER được tính bằng khoảng cách Levenshtein chia cho độ dài chuỗi ground-truth, nên khi mô hình sinh chuỗi lặp dài gấp nhiều lần so với nhãn gốc, số phép chèn (insertion) vượt xa số ký tự đích. Kết quả này phản ánh model collapse của checkpoint hiện tại thay vì kết luận phủ định toàn bộ hướng OCR-free.

Bảng 7.4: Paired bootstrap present-only cho chênh lệch LayoutXLM trừ Baseline, seed 20260620, 50,000 lần lấy mẫu.

Metric	Chênh lệch (điểm %)	Khoảng 95%	$P(\Delta > 0)$ (%)
EM	2,55	[0,80; 4,32]	99,71
NES	8,42	[5,70; 11,22]	100,00

Bảng 7.5: Phân tích độ nhạy present-only sau khi loại bốn mẫu có sai lệch hệ tọa độ ảnh/OCR ($n = 230$).

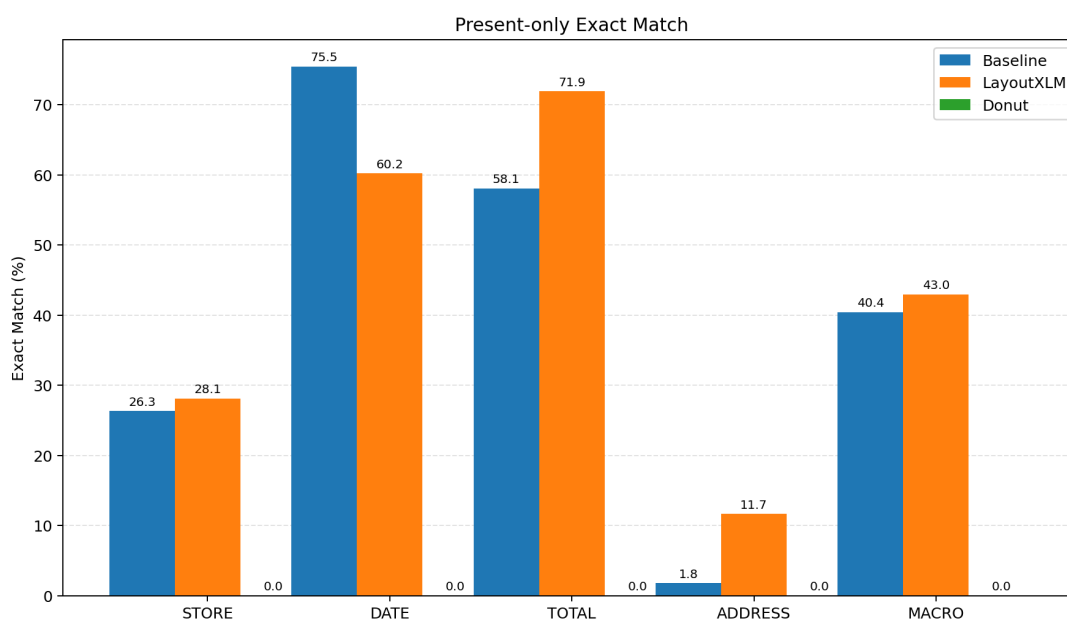
Phương pháp	Macro EM (%)	Macro NES (%)	Macro CER (%)
Baseline	40,94	56,02	56,31
VietOCR + LAYOUTXLM	43,53	64,75	37,98

Paired bootstrap 50.000 lần trên 234 mẫu cho thấy LayoutXLM hơn Baseline +2,55 điểm Macro EM (CI 95%: [0,80; 4,32], $P(\Delta > 0) = 99,71\%$) và +8,42 điểm Macro NES (CI 95%: [5,70; 11,22], $P(\Delta > 0) = 100,00\%$). Tuy nhiên, kết quả này chưa loại trừ hoàn toàn các yếu tố nhiễu từ cấu hình, dữ liệu huấn luyện và cách gán nhãn khác nhau. Khi loại bỏ bốn mẫu bị sai lệch tọa độ, thứ tự hiệu năng của các mô hình vẫn không đổi: LayoutXLM đạt kết quả cao nhất, tiếp theo là Baseline và Donut. Phân tích này nhằm kiểm tra tính ổn định của nhận định, không thay thế cho kết quả đo chính thức trên toàn bộ test set.

7.2. Phân tích theo trường

- **store_name:** LayoutXLM đạt EM present-only 28,12%, cao hơn Baseline 26,34%. NES của LayoutXLM đạt 57,08%, cho thấy nhiều prediction gần đúng nhưng chưa khớp hoàn toàn.
- **date:** Baseline đạt EM present-only 75,46%, cao hơn LayoutXLM 60,19%. Cấu trúc ngày phù hợp với regex, trong khi token classifier có thể chọn nhầm một chuỗi ngày khác trong tài liệu.
- **total:** LayoutXLM đạt EM present-only 71,89%, so với 58,06% của Baseline. Đây là trường LayoutXLM tạo khác biệt rõ nhất trong checkpoint hiện tại.
- **address:** Cả hai phương pháp còn yếu về exact match; LayoutXLM đạt 11,66%, Baseline đạt 1,79%. Tuy nhiên NES của LayoutXLM đạt 60,14%, phản ánh nhiều trường hợp lấy được một phần địa chỉ.

Những nhận xét trên dựa trên kết quả thực tế đo được, chưa đi sâu phân tích nguyên nhân. Để hiểu rõ hơn cơ chế đóng góp của từng thành phần, đồ án sẽ cần thực hiện các thí nghiệm ablation nhằm tách biệt tầm ảnh hưởng của văn bản, bố cục và đặc trưng hình ảnh.



Hình 7.1: So sánh Exact Match trên các mẫu có ground-truth khác rỗng. Hình được sinh từ metrics JSON.

7.3. Latency của các thành phần đã ghi nhận

Bảng 7.6: Latency hậu xử lý từ OCR cache. Phép đo bắt đầu sau khi OCR cache đã sẵn sàng; **chưa bao gồm thời gian chạy PaddleOCR + VietOCR từ ảnh thô.**

Phương pháp	Phạm vi phép đo	Trung bình (ms/ảnh)
Baseline	Hậu xử lý từ OCR cache	1,08
VietOCR + LAYOUTXLM	Suy luận và hậu xử lý từ OCR cache	87,11

Bảng 7.7: Latency suy luận trực tiếp từ ảnh thô. Phép đo này **không so sánh ngang hàng** với bảng post-OCR cache vì phạm vi đo hoàn toàn khác nhau.

Phương pháp	Phạm vi phép đo	Trung bình (ms/ảnh)
DONUT	Suy luận trực tiếp từ ảnh đầu vào	5411,17

Bảng 7.6 trình bày latency của Baseline và LayoutXLM trong phạm vi sau OCR cache — hai phương pháp này có thể so sánh trực tiếp với nhau vì cùng phạm vi đo. **Bảng 7.7** trình bày riêng latency của Donut, đo trực tiếp từ ảnh đầu vào; phạm vi đo này hoàn toàn khác nên hai bảng không so sánh ngang hàng được. Không thể suy ra latency sản xuất từ ảnh thô cho pipeline OCR-based khi thiếu phép đo PaddleOCR + VietOCR cùng điều kiện phân cứng.

Bảng 7.8: Thông tin bổ sung về artifact Donut (generation length analysis).

Quan sát	Giá trị
Trạng thái	Hợp lệ cho so sánh định lượng, diễn giải như ca model collapse
Checkpoint <code>max_length</code> nếu không override	20 token (<code>transformers_default</code>)
Lượt suy luận thực tế	Truyền <code>explicit max_length=768</code> token vào <code>generate()</code>
Nhãn dài hơn fallback metadata	225/234 mẫu; trung vị 47.0, tối đa 104 token
Output đúng bằng fallback metadata	0/234 mẫu
Macro EM all-samples	5,98%
Macro EM present-only	0,00%
Latency artifact	5411,17 ms/ảnh; phép đo bị ảnh hưởng bởi sinh tự hồi quy dài/lặp

7.4. Phân tích hiệu năng Donut và hiện tượng suy sụp

Dù lượt prediction Donut truyền `explicit generation_max_length=768`, Macro EM present-only vẫn ở mức 0,00%. Khi kiểm tra chuỗi sinh thô, mô hình có biểu hiện lặp từ liên tục và không sinh ra cấu trúc thẻ XML hợp lệ. Do đó, kết quả này được đánh giá là một ca thất bại thực nghiệm (negative result) do hiện tượng sụp đổ mô hình (model collapse). Cần lưu ý rằng paper gốc của Donut fine-tune thành công trên tập CORD chỉ khoảng 800 mẫu, nên 1.091 mẫu không quá nhỏ về mặt lý thuyết. Các nguyên nhân kỹ thuật có thể bao gồm:

1. **Thiếu `repetition_penalty`**: Cấu hình sinh (`generation`) trong `donut.yaml` chỉ có `num_beams=2` và `early_stopping=true`, không thiết lập `repetition_penalty`. Đây là nguyên nhân trực tiếp nhất dẫn đến vòng lặp sinh vô tận.
2. **Effective batch size nhỏ**: Batch size hiệu dụng chỉ là $2 \times 8 = 16$ mẫu/step, kết hợp với cosine scheduler chạy 150 epoch, dễ gây gradient nhiễu và khó hội tụ ổn định.
3. **Bug `max_length` mặc định**: Checkpoint không lưu `max_length` vào `generation_config.json` khiến validation loop khi chọn checkpoint có thể bị giới hạn ở 20 token (chỉ đủ cho 3–4 trường). Nếu checkpoint tốt nhất được chọn dựa trên output 20 token bị cắt, mô hình đã bị “dạy sai” mục tiêu từ sớm.
4. **`gradient_checkpointing`**: Cấu hình bật `gradient_checkpointing: true`. Không phải mọi kiến trúc đều hỗ trợ ổn định tính năng này; cần kiểm tra xem nó có gây mất ổn định huấn luyện cho Donut hay không.

Điểm số của Donut ở đây tuyệt đối không được dùng để đưa ra kết luận rằng kiến trúc OCR-free kém hơn kiến trúc dựa trên OCR.

7.5. Phân tích lỗi có thể tái lập

Phân tích lỗi tự động dựa trên phân loại lỗi trong [Bảng 7.9](#). Mỗi kết quả dự đoán sai được gán vào một nhóm lỗi dựa theo bộ quy tắc định sẵn trong mã nguồn. Đây là công cụ thống kê nhất

quán nhằm hỗ trợ phân tích định lượng.

Bảng 7.9: Phân loại lỗi được sinh tự động từ tập kết quả dự đoán.

Loại lỗi	Tiêu chí
EMPTY_PRED	Ground-truth có giá trị nhưng prediction rỗng.
FORMAT_ERROR	Prediction ngày hoặc tổng tiền không đúng schema chuẩn hóa.
OCR_MISS	Với Baseline/LayoutXLM, chuỗi gold không xuất hiện và không có cửa sổ OCR đủ gần.
OCR_WRONG	Với Baseline/LayoutXLM, OCR có cửa sổ gần gold nhưng sai ký tự.
POSTPROCESS_BAD	Prediction thô gần gold nhưng bị thay đổi sai sau chuẩn hóa.
MODEL_BAD	Các trường hợp sai còn lại của luật hoặc mô hình.
LABEL_BAD	Ground-truth rỗng nhưng hệ thống trả về một giá trị.

Với LayoutXLM, EMPTY_PRED, OCR_MISS và MODEL_BAD đều xuất hiện; điều này phù hợp với nhận định rằng pipeline phụ thuộc vào OCR, nhưng taxonomy tự động chưa đủ để định lượng chính xác phần hao hụt do OCR. CSV Donut vẫn cho thấy nhiều prediction rỗng, nhưng phân bố này bị chi phối bởi hiện tượng sinh lặp/model collapse và chỉ có ý nghĩa chẩn đoán artifact.

Các khái niệm hallucination và lỗi BIO phân mảnh chỉ dùng khi có đối chiếu cụ thể với ảnh, chuỗi sinh và token labels.

7.6. Oracle OCR và câu hỏi về ảnh hưởng của OCR

RQ3 yêu cầu đo mức ảnh hưởng của OCR lên LayoutXLM. Thí nghiệm này được thực thi bằng kịch bản suy luận riêng `src/receipt_ie/inference/infer_layoutxlm_oracle.py`, đọc trực tiếp ground-truth OCR từ dataset để thay thế hoàn toàn đường ống OCR. Phép so sánh này chỉ dùng 168 mẫu MC-OCR có sẵn `oracle_ocr`. Subset này phải loại trừ 66 mẫu `self_collected` vì chúng không có OCR hoàn hảo; nếu chấm gộp cả 234 mẫu, các mẫu thiếu oracle sẽ trả rỗng và kéo sai lệch EM trung bình xuống.

Trên subset 168 mẫu này, LAYOUTXLM (Oracle OCR) đạt Macro EM/NES/CER present-only lần lượt là 95,55% / 98,20% / 2,02%, so với 49,72% / 76,26% / 26,09% khi dùng OCR thực tế. Theo từng trường, EM tăng từ 37,97% lên 91,14% ở **store_name**, từ 63,58% lên 100,00% ở **date**, từ 80,79% lên 99,34% ở **total**, và từ 16,56% lên 91,72% ở **address**. Chênh lệch lớn nhất nằm ở **address** và **store_name**, hai trường phụ thuộc mạnh vào nhận dạng chuỗi tiếng Việt dài và nhiều bố cục.

Kết quả này cho thấy OCR là nút thắt cổ chai lớn của pipeline LayoutXLM trên phần dữ liệu có oracle.

Bảng 7.10: So sánh LAYOUTXLM với OCR thực tế và Oracle OCR trên 168 mẫu MC-OCR có ground-truth OCR. Các số liệu là present-only; số mẫu theo bốn trường lần lượt là 158, 151, 151 và 157.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	Macro
Exact Match (EM %) ↑					
VietOCR + LAYOUTXLM	37,97	63,58	80,79	16,56	49,72
LAYOUTXLM (Oracle OCR)	91,14	100,00	99,34	91,72	95,55
Normalized Edit Similarity (NES %) ↑					
VietOCR + LAYOUTXLM	78,03	64,64	87,33	75,04	76,26
LAYOUTXLM (Oracle OCR)	95,63	100,00	99,92	97,24	98,20
Character Error Rate (CER %) ↓					
VietOCR + LAYOUTXLM	25,63	35,36	12,81	30,57	26,09
LAYOUTXLM (Oracle OCR)	4,38	0,00	0,08	3,62	2,02

Giới hạn phạm vi thí nghiệm Oracle OCR

Kết quả 49,72% → 95,55% chỉ đo trên 168/234 mẫu test (72%), toàn bộ thuộc nguồn MC-OCR 2021. 66 mẫu tự thu thập (*self_collected*) bị loại hoàn toàn vì không có ground-truth OCR. Phân phối lỗi OCR trên ảnh tự thu thập — vốn có chất lượng ảnh, font chữ và bố cục khác biệt so với MC-OCR — có thể làm thay đổi đáng kể mức tăng khi áp dụng Oracle. Do đó, kết luận định lượng +45,83 điểm Macro EM **không generalizable** sang toàn bộ 234 mẫu test.

7.7. Demo ứng dụng

Repository có ứng dụng Gradio cho phép chọn phương pháp, tải ảnh lên và xem JSON cùng latency của lượt chạy. Ứng dụng được khởi chạy bằng:

```
python -m receipt_ie.app.gradio_app
```

Demo hỗ trợ kiểm tra trực quan, nhưng không thay thế protocol đánh giá trên test set.

8. Kết luận và hướng phát triển

8.1. Trả lời câu hỏi nghiên cứu

RQ1: LayoutXLM cải thiện như thế nào so với Baseline? Trên 234 mẫu test, LAYOUTXLM đạt Macro EM 42,96% và Macro NES 63,89% ở chế độ present-only, cao hơn Baseline tương ứng 40,41% và 55,47%. Chênh lệch Macro EM chỉ là 2,55 điểm %. Cần nhấn mạnh rằng **đây là so sánh giữa các artifact trong điều kiện đồ án**: LayoutXLM chỉ được huấn luyện trên 784 mẫu (ít hơn 28% so với Donut và Baseline không cần huấn luyện), nên mức chênh lệch 2,55 điểm có thể bị đảo chiều nếu điều kiện dữ liệu đồng nhất hơn. Kết luận phù hợp là LayoutXLM cải thiện vừa phải nhưng ổn định hơn ở các trường cần ngữ cảnh/bổ cục như **total** và **address**; Baseline vẫn mạnh ở **date** nhờ cấu trúc ngày phù hợp với luật regex. Mức tăng Macro EM 2,55 điểm cần được cân nhắc cùng chi phí vận hành: LayoutXLM yêu cầu GPU cho inference, pipeline xử lý OCR cache và bbox, cùng bảo trì checkpoint.

RQ2: Donut OCR-free hoạt động ra sao trên dữ liệu nhỏ? DONUT được suy luận với `generation_max_length=768` nhưng chỉ đạt Macro EM present-only 0,00% và Macro NES 0,08%. Kết quả này là một ca thất bại thực nghiệm (negative result) do hiện tượng sụp đổ mô hình (model collapse), biểu hiện qua các chuỗi sinh lặp vô tận và thiếu cặp thẻ hợp lệ. Cần lưu ý rằng paper gốc Donut fine-tune thành công trên CORD chỉ khoảng 800 mẫu, nên việc quy lỗi hoàn toàn cho “dữ liệu quá nhỏ” là chưa đủ. Các nguyên nhân kỹ thuật có khả năng cao hơn bao gồm: (1) thiếu `repetition_penalty` dẫn đến lặp vô tận, (2) effective batch size nhỏ ($2 \times 8 = 16$) với cosine scheduler 150 epoch gây gradient nhiễu, (3) bug `max_length` mặc định 20 token có thể làm hỏng việc chọn checkpoint qua validation loop, và (4) `gradient_checkpointing` có thể không tương thích ổn định với kiến trúc Donut. Do đó, điểm số của DONUT trong dự án này hoàn toàn không đủ tiêu chuẩn để làm benchmark so sánh, và cũng không đại diện cho khả năng thực sự của hướng tiếp cận OCR-free.

RQ3: OCR ảnh hưởng đến LayoutXLM ở mức nào? Thí nghiệm Oracle OCR được đo trên 168 mẫu MC-OCR có ground-truth OCR, chiếm 72% test set (234 mẫu). Trên subset này, LayoutXLM tăng Macro EM present-only từ 49,72% khi dùng VietOCR thực tế lên 95,55% khi dùng OCR hoàn hảo (+45,83 điểm %). Các trường bị ảnh hưởng nặng nhất là **address** (16,56% → 91,72%) và **store_name** (37,97% → 91,14%). Tuy nhiên, kết luận này **không generalizable** sang toàn bộ 234 mẫu test: 66 mẫu tự thu thập bị loại hoàn toàn vì không có ground-truth OCR, và phân phối lỗi OCR trên nhóm ảnh này có thể khác biệt đáng kể so với MC-OCR.

RQ4: Latency khác nhau ra sao trong phạm vi đo hiện tại? Baseline mất trung bình khoảng 1,08 ms/ảnh từ OCR cache, còn LayoutXLM mất khoảng 87,11 ms/ảnh từ OCR cache. Hai con

số này chấp nhận được trong phạm vi sau OCR cache, nhưng chưa bao gồm thời gian chạy PaddleOCR + VietOCR từ ảnh thô. Donut suy luận trực tiếp từ ảnh mất trung bình 5.411,17 ms/ảnh; chi phí này cao do sinh tự hồi quy dài/lặp trên GPU.

8.2. Các yếu tố ảnh hưởng đến độ tin cậy của kết quả

1. **Nhân rộng và mất cân bằng annotation:** All-samples có thể cho điểm khi cả prediction và ground-truth rỗng; present-only được dùng để kiểm soát, nhưng số mẫu có mặt vẫn khác nhau giữa các trường.
2. **Khác biệt dữ liệu huấn luyện:** Donut dùng toàn bộ dữ liệu JSON, trong khi LayoutXLM chỉ dùng mẫu có hộp bao. Vì vậy phép so sánh phản ánh artifact hiện có hơn là so sánh kiến trúc trong điều kiện dữ liệu hoàn toàn đồng nhất.
3. **Rò rỉ store/template:** Quy trình chia dữ liệu đã loại trùng ảnh theo hash, nhưng chưa chứng minh loại bỏ hoàn toàn rò rỉ theo cửa hàng hoặc template bố cục tương tự giữa train/test.
4. **Sai lệch tọa độ ảnh/OCR:** Báo cáo đã thừa nhận tồn tại bốn mẫu có sai lệch hệ tọa độ ảnh/OCR. Đây không nên chỉ xem là sensitivity check mà cần được theo dõi như một bug dữ liệu/pipeline tiềm tàng. Cần audit toàn bộ đường đi của bbox từ OCR cache đến LayoutXLM dataset/inference để xác nhận scale được áp dụng nhất quán.
5. **Tiêu chí checkpoint lịch sử:** Mã huấn luyện hiện tại đã đổi sang chọn checkpoint theo entity-level F1 và chỉ cho fallback metric khi có `-allow_metric_fallback`. Vì không rerun training trong đợt chốt này, bảng kết quả phải được diễn giải theo artifact đã đóng băng; nếu checkpoint lịch sử được chọn theo loss thì đây là hạn chế lịch sử của artifact.
6. **Giới hạn chuỗi LayoutXLM:** Cấu hình có `doc_stride`, nhưng dataset và inference hiện chưa triển khai sliding window thật; chuỗi dài vẫn bị cắt ở 512 token.
7. **Latency chưa end-to-end cho pipeline OCR-based:** Baseline và LayoutXLM chỉ đo từ OCR cache, chưa cộng thời gian PaddleOCR + VietOCR từ ảnh thô. Donut được đo trực tiếp từ ảnh nên phạm vi không hoàn toàn đồng nhất.
8. **Phạm vi Oracle OCR:** Oracle OCR chỉ có cho 168 mẫu MC-OCR; 66 mẫu tự thu thập không có oracle nên không được dùng để kết luận mức tăng 49,72% $\rightarrow$ 95,55%.
9. **Phân tích lỗi heuristic:** Taxonomy tự động giúp thống kê nhất quán nhưng không chứng minh nguyên nhân cuối cùng; các ca phức tạp vẫn cần đối chiếu ảnh, OCR window và output token-level thủ công.

8.3. Đóng góp và hạn chế

Đồ án cung cấp pipeline dữ liệu thống nhất, ba phương pháp trích xuất, prediction artifact trên cùng test set, evaluator xử lý rõ giá trị rỗng, manifest khóa bằng checksum, taxonomy lỗi tái lập và bảng Oracle OCR subset sinh từ artifact. Hạn chế chính là quy mô dữ liệu nhỏ, annotation không đồng nhất, Donut collapse trong thiết lập hiện tại, bốn mẫu LayoutXLM có sai lệch tọa

độ, chưa có sliding window thật và kết quả định lượng vẫn dựa trên artifact đã đóng băng thay vì rerun training sau khi sửa tiêu chí checkpoint.

8.4. Hướng phát triển

1. Mở rộng Oracle OCR hoặc thiết kế đánh giá bổ sung cho 66 mẫu tự thu thập chưa có ground-truth OCR hoàn hảo.
2. Triển khai sliding windows cho tài liệu dài, gộp prediction giữa các cửa sổ và kiểm tra tác động lên **total/address**.
3. Rerun LayoutXLM với checkpoint selection theo F1, lưu đầy đủ model card gồm commit, môi trường, seed, dữ liệu, thời gian, checkpoint criterion và đường cong huấn luyện.
4. Chạy lại Donut với warm-up/pretraining tốt hơn, repetition penalty và log huấn luyện đầy đủ để kiểm tra liệu model collapse có giảm hay không.
5. Đánh giá split store-held-out/template-held-out và đo latency end-to-end thật từ ảnh thô cho các pipeline OCR-based.

Tài liệu tham khảo

- [1] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer and V. Stoyanov, “Unsupervised Cross-lingual Representation Learning at Scale,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.
- [2] Y. Du, C. Chen, R. Li, Y. Ji, Y. Wu and D. Yu, “PP-OCR: A Practical Ultra Lightweight OCR System,” *arXiv preprint arXiv:2009.09941*, 2020.
- [3] Y. Huang, T. Lv, L. Cui, Y. Lu and F. Wei, “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking,” in *Proceedings of the 30th ACM International Conference on Multimedia 2022*, pages 4083–4091.
- [4] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han and S. Park, “OCR-Free Document Understanding Transformer,” in *European Conference on Computer Vision (ECCV)* Springer, 2022, pages 498–517.
- [5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” *arXiv preprint arXiv:1910.13461*, 2020.
- [6] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang and F. Wei, “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models,” in *Proceedings of the AAAI Conference on Artificial Intelligence* 2023.
- [7] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* 2021, pages 10 012–10 022.
- [8] P. Q. Nguyen and T. Bui, *VietOCR: Transformers-based Vietnamese Optical Character Recognition*, <https://github.com/pbcquoc/vietocr>, 2020.
- [9] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Seo and H. Lee, “CORD: A Consolidated Receipt Dataset for Post-OCR Parsing,” in *Workshop on Document Intelligence at NeurIPS* 2019.
- [10] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei and M. Zhou, “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* 2020, pages 1192–1200.
- [11] Y. Xu, T. Lv, L. Cui, G. Wang, Y. Lu, D. Florêncio, C. Zhang and F. Wei, “LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding,” *arXiv preprint arXiv:2104.08836*, 2021.

- [12] Zalo AI, *MC-OCR 2021: Mobile-Captured OCR Challenge*, <https://ai.zalo.vn/mc-ocr>, 2021.

A. Mẫu dữ liệu JSONL thống nhất

Phụ lục này trình bày cấu trúc dữ liệu của tệp tin JSONL thống nhất nhằm chuẩn hóa đầu vào cho toàn bộ các pipeline xử lý và huấn luyện trong đồ án. Định dạng JSON Lines (JSONL) hỗ trợ lưu trữ mỗi bản ghi biên lai trên một dòng riêng biệt dưới dạng chuỗi JSON, giúp tối ưu hóa bộ nhớ khi đọc ghi luồng dữ liệu lớn.

Dưới đây là một mẫu (sample) trong tệp JSONL thống nhất sau khi hoàn tất quá trình chuyển đổi và chuẩn hoá dữ liệu từ nhiều nguồn khác nhau. Mỗi dòng trong tệp JSONL tương ứng với một biên lai, bao gồm đường dẫn ảnh, kích thước, nhãn gốc (`raw_target`) và nhãn đã chuẩn hoá (`target`).

```

1 {
2   "id": "mcocr_public_145013gguch",
3   "group_id": "mcocr_public_145013gguch",
4   "store_group": "unknown",
5   "source": "mc_ocr_2021",
6   "image_path": "data/raw/mc_ocr_2021/train_images/
   mcocr_public_145013gguch.jpg",
7   "width": 1280, "height": 960,
8   "annotation_level": "json_and_boxes",
9   "raw_target": {
10    "store_name": "MINIMART ANAN",
11    "date": "09/08/2020",
12    "total": "115.000",
13    "address": "Cho Sui Phu Thi Gia Lam"
14  },
15  "target": {
16    "store_name": "MINIMART ANAN",
17    "date": "2020-08-09",
18    "total": "115000",
19    "address": "Cho Sui Phu Thi Gia Lam"
20  }
21 }
```

Đoạn mã 3: Một mẫu JSONL thống nhất

Các trường chính trong mỗi bản ghi:

- **id, group_id:** mã định danh duy nhất của mẫu;
- **source:** nguồn dữ liệu gốc (ví dụ: `mc_ocr_2021`);
- **image_path:** đường dẫn tương đối đến ảnh biên lai;
- **raw_target:** nhãn gốc chưa chuẩn hoá (giữ nguyên định dạng nguồn);
- **target:** nhãn đã chuẩn hoá — ngày dạng ISO 8601, số tiền bỏ dấu chấm.

B. Cấu trúc mã nguồn

Toàn bộ mã nguồn, cấu hình huấn luyện và tệp kết quả dự đoán của đồ án được quản lý tại kho lưu trữ GitHub: <https://github.com/Duc-AnhTp/receipt-vn-ie>. Cây thư mục dưới đây mô tả tổng quan cấu trúc mã nguồn của dự án. Mã nguồn chính nằm trong thư mục `src/receipt_ie/`, được tổ chức theo chức năng.

```

1 receipt-vn-ie/
2 |-- configs/           # YAML configs cho data, model, app
3 |-- data/             # Raw -> Interim -> Processed data pipeline
4 |-- docs/             # Tài liệu hướng dẫn giao diện và quy tắc
5 |-- src/receipt_ie/   # Source code python
6 |   |-- data/         # Convert, normalize, validate, split data
7 |   |-- ocr/          # PaddleOCR (detect) + VietOCR (recognize)
8 |   |-- baseline/     # Heuristics rule-based extractor
9 |   |-- models/       # Wrappers cho Donut và LayoutXLM
10 |  |-- training/      # Scripts huấn luyện model
11 |  |-- inference/     # Pipelines suy luận thời gian thực
12 |  |-- metrics/       # Tính toán EM, NES, CER, latency
13 |  |-- visualization/ # Vẽ bounding box và biểu đồ
14 |  `-- app/          # Giao diện demo Gradio web
15 |-- tests/           # Unit tests (viết bằng pytest)
16 |-- notebooks/       # Sơ tay EDA, phân tích ocr, phân tích lỗi
17 `-- scripts/         # Shell scripts tự động hóa pipeline

```

Đoạn mã 4: Cấu trúc thư mục dự án

C. Cấu hình huấn luyện đầy đủ

Dưới đây là các tệp cấu hình YAML được lưu trữ trong thư mục `configs/`. Với Donut và LayoutXLM, đây là các cấu hình huấn luyện và suy luận tương ứng. Với Baseline, tệp YAML ghi lại các quy tắc heuristic đang áp dụng; các chi tiết đánh giá cụ thể khác được ghi chú riêng ở cuối mục.

C.1. Cấu hình DONUT (`donut.yaml`)

```
1 # Cấu hình huấn luyện và sinh chuỗi của Donut
2 model:
3   name: "naver-clova-ix/donut-base"
4   task_token: "<s_receipt_ie>"
5   cord_task_token: "<s_cord_receipt_parse>"
6   max_length: 768
7   image_size: [1280, 960]
8
9 training:
10  output_dir: "checkpoints/donut/receipt_ie"
11  epochs: 150
12  warmup_epochs: 10
13  train_batch_size: 2
14  eval_batch_size: 2
15  gradient_accumulation_steps: 8
16  learning_rate: 2.0e-5
17  weight_decay: 0.01
18  warmup_ratio: 0.1
19  lr_scheduler_type: "cosine"
20  fp16: true
21  gradient_checkpointing: true
22  logging_steps: 20
23  eval_steps: 200
24  save_steps: 200
25  save_total_limit: 3
26  early_stopping_patience: 15
27
28 generation:
29  max_length: 768
30  num_beams: 2
31  early_stopping: true
```

Đoạn mã 5: Cấu hình huấn luyện DONUT**C.2. Cấu hình LAYOUTXLM (layoutxlm.yaml)**

```
1 # Cấu hình huấn luyện LayoutXLM
2 model:
3   name: "microsoft/layoutxlm-base"
4   max_length: 512
5   doc_stride: 128 # khai báo trong config, chưa dùng sliding window
6   labels:
7     - "O"
8     - "B-STORE_NAME"
9     - "I-STORE_NAME"
10    - "B-DATE"
11    - "I-DATE"
12    - "B-TOTAL"
13    - "I-TOTAL"
14    - "B-ADDRESS"
15    - "I-ADDRESS"
16
17 training:
18   output_dir: "checkpoints/layoutxlm/receipt_ie"
19   epochs: 20
20   train_batch_size: 8
21   eval_batch_size: 8
22   gradient_accumulation_steps: 4
23   gradient_checkpointing: false
24   learning_rate: 2.0e-5
25   weight_decay: 0.01
26   warmup_ratio: 0.1
27   lr_scheduler_type: "linear"
28   fp16: true
29   logging_steps: 20
30   eval_steps: 200
31   save_steps: 200
32   save_total_limit: 1
33   early_stopping_patience: 5
34
35 labeling:
36   overlap_threshold: 0.5
```

Đoạn mã 6: Cấu hình huấn luyện LAYOUTXLM**C.3. Cấu hình luật heuristic (baseline.yaml)**

```
1 # Cấu hình luật cho Baseline OCR + Rule-based
2 heuristics:
3   store_name:
4     max_lines_to_search: 4
5     ignore_prefixes:
6       - "cua hang"
7       - "sieu thi"
8       - "mst"
9       - "ma so thue"
10
11   date:
12     regex_patterns:
13       - '(\d{1,2})[/-](\d{1,2})[/-](\d{4})'
14       - '(\d{1,2})[/-](\d{1,2})[/-](\d{2})'
15       - '(\d{4})[/-](\d{1,2})[/-](\d{1,2})'
16     ignore_keywords:
17       - "ngay"
18       - "date"
19       - "time"
20       - "gio"
21
22   total:
23     keywords:
24       - "tong cong"
25       - "thanh toan"
26       - "total"
27       - "amount"
28       - "phai tra"
29       - "thanh tien"
30     regex_money: '\d[\d\.\,s]*'
31     max_distance_from_keyword: 2
32
33   address:
34     keywords:
35       - "dia chi"
36       - "address"
```

```
37     - "duong"  
38     - "phuong"  
39     - "quan"  
40     - "huyen"  
41     - "tinh"  
42     - "thanh pho"  
43     - "tp"
```

Đoạn mã 7: Cấu hình luật cho phương pháp Baseline**Ghi chú về cấu hình Baseline**

Trong artifact Baseline đã chấm điểm, `infer_baseline.py` gọi trực tiếp `extract_store_name_from_lines` và dùng mặc định `max_lines=5` của hàm này. Vì vậy, mô tả ở chương phương pháp lấy theo đường chạy thực tế của artifact; `baseline.yaml` được giữ trong phụ lục như cấu hình luật lưu trong repo, không phải nguồn tham số duy nhất của lượt suy luận đã báo cáo.

D. Mẫu kết quả trích xuất thực tế

Bảng D.1 trình bày các kết quả được đọc trực tiếp từ prediction artifact hiện có. Các hàng Donut được giữ để minh họa artifact chẩn đoán model collapse trong thiết lập hiện tại, không phải kết luận phủ định toàn bộ hướng OCR-free.

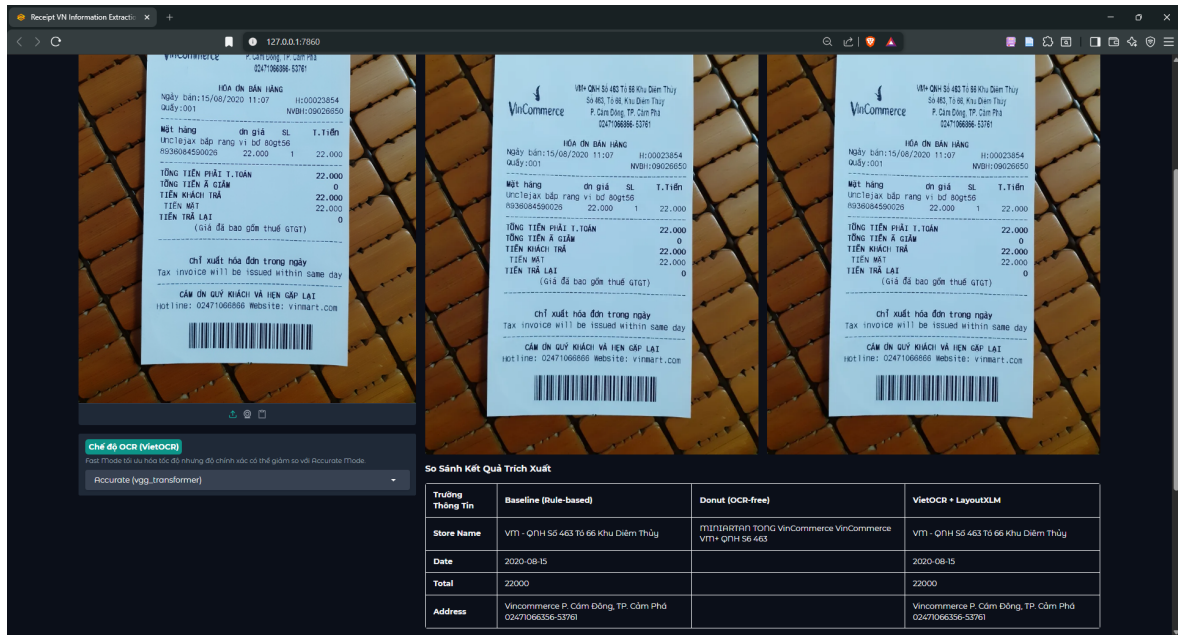
Bảng D.1: Ví dụ kết quả trích xuất thực tế so với nhãn ground-truth trên tập test.

ID	Phương pháp	Trường	Ground-truth	Prediction chuẩn hóa
...145014mbvkd	LAYOUTXLM	total	76000	76000
	Baseline	address	ĐC: 212 Đường Trần Phú-Cẩm Phả	Đc: 212 Đường Trần Phú-Cẩm Phả - Nhân viên thu ngân
	DONUT	store_name	NHÀ SÁCH GD-TC CẨM PHẢ	O cafe O cafe NHA cafe Cafe ...
...145014vauus	LAYOUTXLM	total	19000	19000
	Baseline	total	19000	50000
	DONUT	total	19000	rỗng
...145014aixgf	LAYOUTXLM	date	2020-08-14	2020-08-14
	Baseline	address	ĐC: 212 Đường Trần Phú - Cẩm Phả	Đc: 212 Đường Trần Phú-Cẩm Phả - Nhân viên thu ngân tể

E. Giao diện Demo Gradio

Giao diện ứng dụng Demo Gradio cung cấp một công cụ trực quan hóa giúp nhanh chóng thử nghiệm khả năng nhận dạng và trích xuất của ba phương pháp đối với ảnh biên lai tải lên từ thiết bị cá nhân.

Giao diện demo được xây dựng bằng Gradio, hỗ trợ tải ảnh biên lai, lựa chọn chế độ OCR và hiển thị trực tiếp kết quả so sánh theo từng trường thông tin của Baseline, DONUT và VietOCR + LAYOUTXML. **Hình E.1** là ảnh chụp từ một lượt chạy thực tế trên máy cục bộ.



Hình E.1: Giao diện Gradio trong một lượt chạy thực tế, so sánh kết quả của Baseline, DONUT và VietOCR + LAYOUTXML.

E.1. Hướng dẫn cài đặt và chạy demo

Để cài đặt và khởi chạy ứng dụng demo, thực hiện các bước sau:

```

1 # Cài đặt project ở chế độ editable
2 pip install -e .
3 # Cài đặt các thư viện cho ứng dụng demo
4 pip install -r requirements/app.txt
5 # Khởi chạy ứng dụng demo (sử dụng module entry point)
6 python -m receipt_ie.app.gradio_app

```

Đoạn mã 8: Cài đặt và chạy ứng dụng Gradio demo

Sau khi chạy lệnh trên, ứng dụng sẽ khởi động tại địa chỉ `http://localhost:7860`. Người dùng có thể truy cập qua trình duyệt web để sử dụng giao diện demo trực quan.